The software-evolution phase is complete and frozen.

Do not modify the DaPlanStan application source.

Use the final verified Scenario 3 inventory:

Controller (GRASP) = 9 instances
Information Expert (GRASP) = 1
Observer (GoF) = 2
Adapter (GoF) = 1
Strategy (GoF) = 1

Total = 14 confirmed instances
Distinct pattern types = 5

Faculty requirement:

For EACH detected pattern type, select one representative confirmed pattern instance and implement the same small problem in:

- Java
- Python
- JavaScript
- C++

Therefore the current written requirement implies:

5 representative pattern problems × 4 languages = 20 implementations.

Do not write cross-language code yet.

First prepare the cross-language selection and common specifications.

1. Read the final verified pattern records and cross-language handoff.

2. Select exactly ONE representative confirmed instance for each pattern type:

- Controller (GRASP)
- Information Expert (GRASP)
- Observer (GoF)
- Adapter (GoF)
- Strategy (GoF)

3. Prefer representative instances that:

- are clearly source-supported
- have a small, understandable problem
- can be implemented consistently in all four languages
- do not depend on Rails, Anthropic, Turbo, databases or external APIs
- preserve the core responsibility/pattern collaboration
- can be tested with identical inputs and expected outputs

4. For each pattern type, compare the available confirmed instances before choosing.

For Controller, choose the cleanest representative among the confirmed Controller collaborations rather than automatically choosing the newest one.

For Observer, compare:
- Outbound Mail Delivery Logging
- Trip-Scoped Turbo Refresh Notifications

For Adapter use the verified:
- Multiple Booking Provider Normalization

For Strategy use:
- Cancellation Policy Evaluation

For Information Expert use:
- Parsed Email to Trip Record Linking
unless another verified Information Expert exists. Do not invent one.

5. For every selected representative, state:

- pattern type
- selected source instance
- source pattern ID
- original DaPlanStan problem
- simplified cross-language problem
- why it is representative
- roles that must remain visible in all four implementations

6. Design one language-neutral specification for each selected pattern.

Each specification must define:

- input
- output
- business rules
- error cases
- deterministic expected behavior
- shared fixtures/test cases

The four language implementations for a given pattern must use the SAME logical problem, same inputs and same expected outputs.

7. Keep the snippets deliberately small.

Do not reproduce the whole Rails application.

The goal is to demonstrate the same pattern collaboration across Java, Python, JavaScript and C++.

8. For Adapter:

Preserve the core idea of two differently shaped provider inputs normalized into one common output contract.

Use deterministic local data only.

No external API.

9. For Strategy:

Preserve the same Flexible, Moderate and Strict cancellation-policy behavior.

Use the same fixed time boundaries and monetary example across all four languages.

Do not use wall-clock time in tests.

10. For Observer:

Choose a small event/notification problem where subject/observer collaboration remains explicit.

If the original Rails/Turbo implementation relies heavily on framework behavior, simplify the problem without changing the Observer responsibility.

11. For Information Expert:

Preserve the GRASP principle that the object holding the necessary information performs or owns the relevant responsibility.

Do not transform it into another GoF pattern merely to make implementation easier.

12. For Controller:

Preserve the GRASP Controller responsibility of receiving a system event and coordinating domain collaborators.

Do not treat language/framework entry-point syntax as the pattern itself.

13. Define the exact folder plan.

Use a structure suitable for the faculty repository such as:

```
Part3/
  controller/
    java/
    python/
    javascript/
    cpp/
  information_expert/
    java/
    python/
    javascript/
    cpp/
  observer/
    java/
```

```
    python/
    javascript/
    cpp/
  adapter/
    java/
    python/
    javascript/
    cpp/
  strategy/
    java/
    python/
    javascript/
    cpp/
```

Each language folder will later contain:

- production snippet
- test runner/test file
- README
- raw execution result

Do not create them yet.

14. Define the cross_language.csv plan using the faculty fields:

pattern_name
instance_name
language
snippet_path
source_loc
max_function_cc

runtime_compiler
test_command
test_result
median_process_ms
language_notes

Do not fill execution-dependent values yet.

15. Define the measurement protocol.

Use the same rules across languages:

- production LOC excludes blank lines, comments and tests
- record maximum function cyclomatic complexity
- compile first where applicable
- one warmup
- five measured executions
- report median process time
- record runtime/compiler version
- never estimate missing execution times

16. Identify which languages/tools must eventually be available locally:

- Java
- Python
- Node.js
- C++ compiler

Do not claim any runtime is installed unless actually checked.

17. Produce a final selection table with exactly five rows:

Pattern type
Selected source ID
Selected instance
Simplified problem
Why selected

18. Produce five common-specification sections, one per selected pattern.

19. Produce the planned test-case table for each pattern.

20. Do not write implementation code yet.
Do not create cross_language.csv yet.
Do not run benchmarks yet.

At the end give me:

1. Final five representative selections.
2. Justification for each.
3. Five language-neutral problem specifications.
4. Shared test cases for each problem.
5. Required roles/participants to preserve.
6. Folder/file naming plan.
7. Measurement/testing protocol.
8. Any concern about faithfully representing a GRASP pattern outside Rails.
9. Exact implementation order recommended for the next prompts.

Worked for 3m 56s

DaPlanStan — Part 3: Cross-Language Selection and Common Specifications

Selection and specifications finalized

Design-only stage | Source version: `scenario3_cancellation_policies` | 11 October 2026

I reviewed the final `patterns_evolution.csv`, `pattern_counts.csv`, cross-language handoff, verified participant mappings and relevant Scenario 3 source.

The faculty requirement is confirmed: **one representative instance for each of the five detected pattern types, implemented in all four languages.**

Selected pattern problems
5
One per confirmed type

Required implementations
20
Java, Python, JavaScript, C++

No application source, cross-language implementation, fixture file, benchmark or `cross_language.csv` was created or changed.

1. Final five representative selections

Pattern type	Selected ID	Selected confirmed instance	Simplified problem	Why selected
Controller (GRASP)	**P03**	Trip Creation Coordination	Create a trip, register its owner and queue a follow-up task	Small system-event coordination with clear collaborators
Information Expert (GRASP)	**P01**	Parsed Email to Trip Record Linking	Match parsed booking data to an existing or new trip item	Demonstrates responsibility assigned to the information holder
Observer (GoF)	**P08**	Outbound Mail Delivery Logging	Notify a registered logger after simulated message delivery	Explicit registration and callback behavior
Adapter (GoF)	**S2-N01**	Multiple Booking Provider Normalization	Normalize Atlas and Beacon data into one offer format	Demonstrates adaptation of incompatible provider schemas
Strategy (GoF)	**S3-N01**	Cancellation Policy Evaluation	Calculate refunds using interchangeable cancellation policies	Three distinct algorithms sharing one contract

These five names and IDs come from the frozen inventory. No new pattern instance is being invented for Part 3.

2. Why these five were chosen

Controller comparison

All nine confirmed Controller instances were considered.

ID	Collaboration	Cross-language suitability
P02	Inbound Travel Email Intake	Moderate — mailbox routing and sender authorization add complexity
P03	**Trip Creation Coordination**	**High — small, understandable coordination flow**
P04	Trip Membership Management	Moderate — multiple roles and invitation conditions
P05	Bulk Itinerary Import	Moderate — bulk validation, duplicates and callback behavior
P06	OAuth Sign-in and Identity Linking	Low — authentication lifecycle is integration-heavy
P07	Account Traveler Enrollment	Moderate — identity matching and invitation branching
S1-N01	Personalized Itinerary Generation	Low — Anthropic generation and signed preview complexity
S2-N02	Booking Provider Option Coordination	Moderate — would overlap substantially with the Adapter example
S3-N02	Cancellation Quote Coordination	Moderate — would overlap with the Strategy example

P03 is selected because its genuine source responsibility is compact: `TripsController#create` creates a Trip, establishes an owner membership and delegates cover-image retrieval.

Verified locations include `app/controllers/trips_controller.rb:35-48`, `app/models/trip.rb:4-19,31-37`, and `app/models/trip_member.rb:1-6`.

Observer comparison

Confirmed instance	Advantages	Disadvantages
P08 — Outbound Mail Delivery Logging	Explicit `register_observer` and `delivered_email` callback; easy to simulate	Original publisher/registration mechanism is framework-mediated

P09 — Trip-Scoped Turbo Refresh Notifications	Rich subscription and event example	Requires substantial simplification of Turbo, Action Cable and browser subscriptions

P08 is selected. It preserves the crucial idea that a publisher notifies a registered observer without mail-producing code directly invoking the logger.

Source: `config/initializers/mail_logger.rb:1–7` .

Adapter and Strategy each have exactly one confirmed instance, so S2-N01 and S3-N01 are selected directly. Information Expert likewise has only P01.

3. Common execution contract for all five problems

To make results comparable across languages, I recommend one small shared fixture protocol.

Use UTF-8 tab-separated files (`.tsv`) for test inputs and expected results. Each case has a stable `case_id` . Each implementation processes the same fixtures and prints deterministic output. Error cases produce named error codes rather than language-specific exception text.

For all five problems:

- Each test case is independent unless its specification explicitly describes a sequence.
- A literal `~` denotes an absent optional value.
- Fields contain no tab or newline characters.
- Data comparisons and case-insensitive matching use ASCII rules for this small fixture set.
- Successful outputs are compared field by field, in a fixed order.
- Tests compare actual results against one shared expected-results file, not different expected answers per language.
- No external network, database, Rails runtime or live clock is required.

Java, Python, JavaScript and C++ may differ in syntax and type systems, but their **business rules, fixture values and expected outputs must remain identical**.

All prices will be represented in integer cents in the cross-language fixtures, avoiding inconsistent floating-point arithmetic. This is a simplified, exact representation of the monetary behavior in the source.

4. Common specification A — Controller (GRASP)

Selected P03: Trip Creation Coordination

Original problem

`TripsController#create` receives a trip-creation request, sets the current user and account, saves a Trip, creates its owner's `TripMember`, and schedules cover-image retrieval.

Simplified problem

A user requests a new trip. A `TripCreationController` validates the request, delegates Trip creation to domain objects, creates the initial owner membership, and records a simulated cover-task request.

There is no database or image API.

Required participants

Role	Cross-language participant
System-event coordinator	`TripCreationController`
Domain entity	`Trip`
Account/owner context	`Account`
Membership entity	`TripMember`
In-memory persistence collaborator	`TripStore`
Simulated asynchronous collaborator	`CoverTaskQueue`

The Controller must coordinate these participants rather than implement every domain responsibility itself.

Inputs

Field	Type
`case_id`	String

`user_id`	String or ~
`owner_traveler_id`	String or ~
`title`	String
`start_date`	ISO date
`end_date`	ISO date

Business rules

The user must be authenticated and have an owner traveler. The title must not be blank after trimming. Both dates must be valid Gregorian dates, and the end date cannot precede the start date.

On success, create one Trip, one owner-role TripMember and one simulated cover-task request. On failure, create none of them.

For deterministic output, the new Trip ID is `trip-` followed by the case ID.

Output

Success:

```
Plain text
case_id    CREATED    trip_id    normalized_title    owner_traveler_id    owner    COVER_QUEUED
```

Failure:

```
Plain text
case_id    ERROR    error_code
```

Shared test cases

ID	Input condition	Expected result
C01	Valid trip, 16–18 November 2026	`CREATED` , owner assigned, cover queued
C02	`user_id = ~`	`ERROR UNAUTHENTICATED`
C03	`owner_traveler_id = ~`	`ERROR MISSING_OWNER`
C04	Title contains only spaces	`ERROR MISSING_TITLE`
C05	Start date `2026-02-30`	`ERROR INVALID_DATE`
C06	End date earlier than start	`ERROR INVALID_DATE_RANGE`
C07	Start and end are the same valid day	`CREATED`
C08	Title `"  City Break  "`	`CREATED` , title `City Break`

The unit tests must also confirm that invalid requests produce no membership or queued cover task.

Pattern invariant: The Controller receives the system event and delegates domain creation, membership creation and task scheduling. Merely having a function named `main` does not demonstrate GRASP Controller.

5. Common specification B — Information Expert (GRASP)

Selected P01: Parsed Email to Trip Record Linking

Original problem

`InboxItem` owns the extracted email data and handles linking it to existing or new `TripItem` records through `fill_missing_links!` .

Source: `app/models/inbox_item.rb:19-25,67-187` .

Simplified problem

An `InboxItem` contains a parsed booking provider name and confirmation reference. It examines itinerary records belonging to its trip.

If it finds a matching item by case-insensitive name, it fills missing link/reference fields. Otherwise, it creates a new confirmed itinerary record.

Required participants

Role	Cross-language participant
Information Expert	`InboxItem`
Information-holding fields	Parsed provider name, reference and trip ID
Domain record	`TripItem`
Caller	`InboxReviewController` or simple request handler
Collection context	`Trip` with its items

Matching and linking logic must belong to `InboxItem`, not be moved entirely into the caller.

Inputs

`case_id`, `inbox_id`, `trip_id`, `provider_name`, `incoming_reference`, `existing_item_id`, `existing_trip_id`, `existing_item_name`, `existing_reference`, `existing_inbox_id`.

An absent existing item is represented by `~` in its fields.

Business rules

1. Match existing items only within the same trip.
2. Compare names case-insensitively.
3. If a matching record exists, reuse it.
4. Populate its missing confirmation reference and inbox link.
5. Do not overwrite existing nonblank values.
6. If no match exists, create a new confirmed TripItem linked to the InboxItem.
7. Reject a blank provider name.

8. Never link an item from another trip.

Output

```
Plain text
case_id    LINKED|CREATED    item_id    item_name    confirmation_ref    inbox_id
```

New record IDs use `item-` followed by the case ID. The `~` value represents missing references.

Shared test cases

ID	Test	Expected result
I01	Matching item; no existing reference	`LINKED` ; missing reference filled
I02	No existing item	`CREATED` ; new confirmed item
I03	Matching item already has reference	`LINKED` ; original reference retained
I04	Same name exists in another trip	`CREATED` in current trip
I05	Provider name is missing	`ERROR MISSING_PROVIDER`
I06	Case difference in provider/item name	`LINKED`
I07	Matching item already linked to another inbox	Existing inbox link preserved
I08	Incoming reference missing	Item linked/created with missing reference

Pattern invariant: The object with the relevant parsed booking information owns the decision about linking. This demonstrates GRASP Information Expert; it is not a Factory or Adapter example.

6. Common specification C — Observer (GoF)

Selected P08: Outbound Mail Delivery Logging

Original problem

DaPlanStan registers `MailLogger` as an Action Mailer observer. When an outbound message is delivered, the mail framework invokes `MailLogger.delivered_email`.

Simplified problem

A local `MailDeliveryPublisher` simulates successful or failed delivery. A `MailLogger` can register to receive successful-delivery notifications.

Nothing is actually emailed.

Required participants

Observer role	Cross-language participant
Subject/publisher	`MailDeliveryPublisher`
Observer contract	`onDelivered(message)`
ConcreteObserver	`MailLogger`
Event	`MailMessage`
Registration	`register / unregister` mechanism

The publisher must invoke observers through their registered callback interface. It must not directly depend on `MailLogger` as a special-case logger.

Inputs

Field	Type
`case_id`	String
`register_logger`	`YES` or `NO`
`delivered`	`YES` or `NO`

`message_id`	String
`subject`	String
`recipient`	String

Each case constructs a fresh publisher.

Outputs

If the logger receives a successful-delivery event:

```
Plain text

case_id    LOGGED    message_id    subject    recipient
```

Otherwise:

```
Plain text

case_id    NO_NOTIFICATION
```

Shared test cases

ID	Condition	Expected result
O01	Registered and delivered	`LOGGED` with correct details
O02	Not registered, delivered	`NO_NOTIFICATION`
O03	Registered, delivery failed	`NO_NOTIFICATION`
O04	Registered, different valid message	`LOGGED` with new message details

O05	Delivered with missing recipient	`ERROR INVALID_RECIPIENT`
O06	Invalid registration flag	`ERROR INVALID_FLAG`
O07	Observer unregistered before delivery	`NO_NOTIFICATION`

For O07, the test runner will exercise `register → unregister → deliver` as a fixed sequence.

Pattern invariant: Subject and observer remain separately identifiable, and notification depends on subscription. A direct `MailLogger.log()` call by the sending code would not meet this specification.

7. Common specification D — Adapter (GoF)

Selected S2-N01: Multiple Booking Provider Normalization

Original problem

DaPlanStan's Atlas and Beacon providers expose differently shaped local booking-offer data.

`AtlasAdapter#offers_for` and `BeaconAdapter#offers_for` translate their formats into the same application-facing structure.

Source: `app/services/booking_providers/registry.rb:11–66`, `atlas_adapter.rb:4–35`, and `beacon_adapter.rb:4–33`.

Simplified problem

Two local provider catalogues expose incompatible field names, layouts and price representations.

The application selects a provider and receives a normalized offer record, independent of the source format.

Required participants

Adapter role	Cross-language participant
Client	`BookingOfferClient`
Target	Common `offersFor(destination)` contract

Adapter 1	`AtlasAdapter`
Adapter 2	`BeaconAdapter`
Adaptees	Atlas and Beacon raw data structures
Selection	`ProviderRegistry`

The raw fixtures represent the adaptees. No artificial `Adaptee` class is necessary.

Different raw formats

Atlas

Destination-scoped records

`id`, `title`, `category`, `location`, `details` and a compound price such as `EUR:24.00`.

Beacon

Flat records with destination field

`code`, `label`, `type`, `address`, `summary`, `amount` and `currency` as separate fields.

To keep Java and C++ dependency-free, Part 3 may represent these two structures using **different TSV layouts** rather than requiring external JSON libraries. The field names, price composition and record layout must still differ.

Common normalized output

Normalized field	Example
`provider_code`	`atlas_demo`
`provider_name`	`Atlas Demo`
`offer_id`	`atlas-rome-walk-01`
`name`	`Ancient Rome Walking Sampler`
`kind`	`do`
`address`	`Rome, Italy`
`description`	Fictional guided walk description

`sample_amount_cents`	`2400`
`currency`	`EUR`

Business rules

- Only `atlas_demo` and `beacon_demo` are valid providers.
- Only `rome` and `paris` are supported demo destinations.
- The source offer must belong to the requested provider and destination.
- Convert provider categories into valid itinerary kinds.
- Normalize currency and amount without rounding ambiguity.
- Reject missing IDs, invalid kinds, negative amounts and malformed provider records.
- Do not generate a reservation or confirmation reference.

Shared test cases

ID	Input	Expected result
A01	Atlas, Rome, walking sampler	`atlas_demo` , `do` , 2400 cents, EUR
A02	Beacon, Rome, museum sampler	`beacon_demo` , `do` , 1600 cents, EUR
A03	Atlas, Paris, market browsing	`shop` , 0 cents
A04	Beacon, Paris, pastry stop	`eat` , 1250 cents
A05	Unknown provider code	`ERROR UNKNOWN_PROVIDER`
A06	Unsupported destination	`ERROR UNKNOWN_DESTINATION`
A07	Atlas selected with Beacon offer ID	`ERROR UNKNOWN_OFFER`
A08	Source entry with unknown category	`ERROR INVALID_OFFER`
A09	Source entry with malformed price	`ERROR INVALID_OFFER`
A10	Fixture missing required structure	`ERROR INVALID_FIXTURE`

The fixture set will include the four valid offers above and separate malformed variants for A08–A10. The same fixture versions must be used by every language.

Pattern invariant: The client uses only the common target interface. Atlas-specific and Beacon-specific field interpretation stays inside the corresponding adapters.

8. Common specification E — Strategy (GoF)

Selected S3-N01: Cancellation Policy Evaluation

Original problem

`CancellationPolicies::QuoteEvaluator` selects one of three policy objects and delegates the variable fee-rate calculation through `fee_rate(seconds_until_start:)`.

Simplified problem

Calculate a hypothetical cancellation fee and refund using Flexible, Moderate or Strict rules.

No booking is cancelled, and no real provider is contacted.

Required participants

Strategy role	Cross-language participant
Context	`QuoteEvaluator`
Strategy interface	`feeRate(secondsUntilStart)`
ConcreteStrategy	`FlexiblePolicy`
ConcreteStrategy	`ModeratePolicy`
ConcreteStrategy	`StrictPolicy`
Client	Test runner / quote-request handler

Policy selection must occur through an approved list of codes. The Context delegates calculation to the selected policy object.

Inputs

Field	Type
`case_id`	String
`policy_code`	flexible, moderate, strict
`seconds_until_start`	Signed integer
`amount_cents`	Nonnegative integer
`currency`	EUR, GBP or USD

The integer time difference replaces dependence on a live clock. It directly reflects the source's `seconds_until_start` policy contract.

For this deliberately small snippet, the booking is assumed to have already passed the full application's email-link and eligibility checks. The demonstration concentrates on the verified Strategy collaboration.

Frozen policy rules

Time until start	Flexible	Moderate	Strict
At least 72h	0%	0%	100%
At least 24h but less than 72h	0%	50%	100%
More than 0 but less than 24h	10%	100%	100%
At/after start	100%	100%	100%

Calculate in integer cents, rounding fractional cents half-up:

$$\text{fee cents} = \left\lfloor \frac{\text{amount cents} \times \text{fee percentage} + 50}{100} \right\rfloor$$

$$\text{refund cents} = \text{amount cents} - \text{fee cents}$$

This represents the same two-decimal monetary behavior as the source's BigDecimal calculation for the supported positive monetary inputs.

Output

Plain text

```
case_id    OK    policy_code    fee_cents    refund_cents    currency    refundable
```

`refundable` is `YES` only when the refund is greater than zero.

Shared test cases

ID	Policy	Seconds until start	Amount	Expected fee/refund
S01	Moderate	259200 (72h)	€200	€0 / €200
S02	Moderate	259199	€200	€100 / €100
S03	Moderate	86400 (24h)	€200	€100 / €100
S04	Moderate	86399	€200	€200 / €0
S05	Flexible	86400	€200	€0 / €200
S06	Flexible	86399	€200	€20 / €180
S07	Flexible	1	€200	€20 / €180
S08	Flexible	0	€200	€200 / €0
S09	Strict	345600 (96h)	€200	€200 / €0
S10	Moderate	172800 (48h)	€0.05	€0.03 / €0.02

S11	Strict	100	€0	€0 / €0; not refundable
S12	Unknown policy	Any valid time	€200	`ERROR UNKNOWN_POLICY`
S13	Flexible	Valid	Negative amount	`ERROR INVALID_AMOUNT`
S14	Flexible	Valid	JPY amount	`ERROR UNSUPPORTED_CURRENCY`
S15	Flexible	Missing time	€200	`ERROR INVALID_TIME`

These fixtures cover the exact deadlines, different policy outcomes, rounding, invalid values and the zero-amount edge case.

Pattern invariant: Three interchangeable policy objects provide variable algorithms. A single large switch containing all policy fee rules would fail the intended Strategy demonstration.

9. Folder and naming plan

No directories or files have been created yet. This is the exact recommended structure.

Plain text

```
Part3/
├── common_specs/
│   ├── README.md
│   ├── controller_cases.tsv
│   ├── controller_expected.tsv
│   ├── information_expert_cases.tsv
│   ├── information_expert_expected.tsv
│   ├── observer_cases.tsv
│   ├── observer_expected.tsv
│   ├── adapter_cases.tsv
│   ├── adapter_expected.tsv
│   ├── atlas_raw.tsv
│   ├── beacon_raw.tsv
│   ├── adapter_invalid_fixtures/
│   ├── strategy_cases.tsv
```

```
│   └── strategy_expected.tsv
├── controller/
│   ├── java/
│   ├── python/
│   ├── javascript/
│   └── cpp/
├── information_expert/
│   ├── java/
│   ├── python/
│   ├── javascript/
│   └── cpp/
├── observer/
│   ├── java/
│   ├── python/
│   ├── javascript/
│   └── cpp/
├── adapter/
│   ├── java/
```

Files planned inside each language folder

Language	Production	Test runner	Other required files
Java	`Main.java`	`TestMain.java`	`README.md`, `test_result.txt`, `timings.csv`
Python	`main.py`	`test_main.py`	Same
JavaScript	`main.js`	`test_main.js`	Same
C++	`main.cpp`	`test_main.cpp`	Same

Production classes may live in more than one source file if necessary to preserve meaningful roles. Every production file will be included in the LOC and complexity calculation. Tests and generated binaries will be excluded.

A shared `common_specs/README.md` will document fixture formats, expected outputs, parsing rules and any controlled simplifications.

10. Faculty `cross_language.csv` plan

The faculty requires the following exact columns:

Column	Planned value
`pattern_name`	Controller, Information Expert, Observer, Adapter or Strategy
`instance_name`	Exact stable source-instance name from the frozen records
`language`	Java, Python, JavaScript or C++
`snippet_path`	Relative path to the production source
`source_loc`	Measured production LOC
`max_function_cc`	Measured maximum function cyclomatic complexity
`runtime_compiler`	Actual name and version
`test_command`	Executed command and working-folder context
`test_result`	Actual passed/total or failed result
`median_process_ms`	Measured median of five process runs
`language_notes`	Role mapping and language-specific structure

The final CSV will have **20 data rows**, assuming all five patterns are implemented in all four languages.

It is not being generated now because the source paths, measured values and actual test results do not yet exist.

11. Measurement and verification protocol

Source LOC

Count nonblank, noncomment lines in production files only. Exclude unit tests, fixture data, generated code and compiler artifacts.

Every language must use the same inclusion rule, with any language-specific parsing limitations documented.

Cyclomatic complexity

For consistent reporting, use a declared McCabe-style counting rule:

Start at 1 per function or method, then add decision points for conditional branches, loops, switch cases, exception handlers, ternary decisions and short-circuit Boolean decisions.

Use one agreed counting tool or measurement script for all languages. Report the maximum function complexity for each implementation, not the total complexity of the application.

Timing

Compile Java and C++ before timing. The timing unit should be one fresh process executing the complete fixed fixture set, including process startup, input reading, processing and output.

Run one warm-up, followed by five measured executions on the same machine. Report the median, retain raw timings and never infer missing values.

All twenty implementations must produce the same expected output for their respective shared problem before timing measurements are accepted.

Runtime/tool requirements

Language	Required tools	Status in current workspace
Java	JDK with `java` and `javac`	Executables found on PATH
Python	`python3`	Found on PATH
JavaScript	Node.js	Found on PATH
C++	`g++` or compatible compiler	`g++` found on PATH

These executable paths were checked in this session. Compiler versions, successful compilation of Part 3 snippets and performance have **not** been verified.

The teammate must record actual tool versions and repeat environment checks on the machine used for the final measurements.

12. Representing GRASP faithfully outside Rails

GRASP Controller and Information Expert are **responsibility-assignment patterns**, not syntax-dependent patterns.

For Controller, the cross-language implementation must show a clear request or system-event boundary and meaningful delegation to domain collaborators. Merely moving all logic into a class named `Controller` would be misleading.

For Information Expert, the class possessing the necessary data should own the relevant decision. A language translation that puts all matching behavior into a separate utility and leaves `InboxItem` as a passive data container would no longer faithfully demonstrate the selected collaboration.

Similarly, Observer must preserve registration and event notification, Adapter must preserve interface adaptation, and Strategy must preserve interchangeable algorithms.

The four language versions should express equivalent designs without forcing artificial inheritance where a language naturally uses interfaces, protocols, composition or duck typing.

13. Recommended implementation order

1. **Freeze common fixtures and expected results**
   Create all five shared fixture specifications and one common assertion/output comparison convention before coding.
2. **Strategy — S3-N01**
   Implement Flexible, Moderate and Strict in Java, Python, JavaScript and C++; verify identical monetary outputs and deadline boundaries.
3. **Observer — P08**
   Implement simulated delivery publishing, registration and logging in all four languages; verify event-driven notification.
4. **Controller — P03**
   Implement trip-creation event coordination, owner membership and fake task queue in all four languages.
5. **Information Expert — P01**
   Implement InboxItem-owned booking matching and linking in all four languages; verify existing/new record cases.
6. **Adapter — S2-N01**

Implement Atlas and Beacon normalization in all four languages using the shared local fixture data and validation rules.

7 **Perform full cross-language verification**

Check every fixture and expected output, inspect actual participant structures and record pass/fail results.

8 **Measure and complete faculty artifacts**

Measure LOC, cyclomatic complexity and process timings; prepare 20-row cross_language.csv and Part 3 report.

Final Part 3 readiness

Requirement	Status
Exactly five representative instances selected	✓
Selections grounded in frozen records	✓
Same language-neutral problem per type defined	✓
Common inputs, outputs and error cases specified	✓
Shared test cases planned	✓
Pattern participant roles mapped	✓
Folder structure planned	✓
Measurement procedure defined	✓
Cross-language code written	No
Benchmarks executed	No
`cross_language.csv` created	No
DaPlanStan source modified	No

Part 3 selection and common specifications are complete. The next prompt should begin by materializing the shared fixtures and implementing **Strategy S3-N01** across the four languages, using the fixed test cases above.

Now begin the cross-language implementation phase.

Implement ONLY the first selected pattern:

S3-N01 — Strategy (GoF)
Cancellation Policy Evaluation

Do not implement Controller, Information Expert, Observer or Adapter yet.

Use the frozen common specification exactly as defined in the previous step.

Create the Part3 working structure if it does not yet exist.

1. First create and freeze the shared Strategy fixture files under:

Part3/common_specs/

Create:

strategy_cases.tsv
strategy_expected.tsv

Use the exact frozen cases S01–S15.

Do not alter the business rules or expected results.

The Strategy rules are:

Flexible:
- >= 72 hours → 0% fee

- >= 24 and < 72 hours → 0%
- > 0 and < 24 hours → 10%
- at or after start → 100%

Moderate:
- >= 72 hours → 0%
- >= 24 and < 72 hours → 50%
- > 0 and < 24 hours → 100%
- at or after start → 100%

Strict:
- always 100%

Use integer cents.

Fee calculation must use half-up rounding.

Refund = amount - fee.

Supported currencies:
EUR
GBP
USD

Error codes must be deterministic.

2. Implement the same Strategy problem in exactly four languages.

Create:

Part3/strategy/java/

Part3/strategy/python/
Part3/strategy/javascript/
Part3/strategy/cpp/

Each implementation must preserve the same roles:

Context:
QuoteEvaluator

Strategy contract:
feeRate(secondsUntilStart)

Concrete strategies:
FlexiblePolicy
ModeratePolicy
StrictPolicy

Client:
test runner / quote request handler

Do not replace the three strategies with one giant switch containing all fee algorithms.

Policy selection may use a small approved lookup or switch in the Context.

3. Java implementation.

Create production source and tests.

Prefer simple dependency-free Java.

The production implementation must expose clearly identifiable:

- strategy interface
- FlexiblePolicy
- ModeratePolicy
- StrictPolicy
- QuoteEvaluator

Read the common TSV fixture.

Run all S01–S15.

Write deterministic output.

4. Python implementation.

Use the same logical structure.

Do not collapse the pattern into one function merely because Python permits it.

Keep separate policy objects/classes and a QuoteEvaluator context.

Use no third-party packages.

5. JavaScript implementation.

Use Node.js only.

Keep the same Strategy roles visible.

Use integer arithmetic.

Do not use browser APIs or npm packages.

6. C++ implementation.

Use standard C++ only.

Keep a common Strategy interface with interchangeable concrete implementations.

Use integer arithmetic and no external libraries.

7. Use exactly the same fixtures and expected results in all four languages.

The implementations must not contain separate hard-coded expected answers.

Each test runner must read the shared fixture files and compare actual results against:

strategy_expected.tsv

8. Cover all frozen cases:

S01 Moderate exactly 72h
S02 Moderate one second below 72h
S03 Moderate exactly 24h
S04 Moderate one second below 24h
S05 Flexible exactly 24h
S06 Flexible one second below 24h
S07 Flexible one second before start
S08 Flexible exactly at start
S09 Strict at 96h
S10 Moderate €0.05 rounding case
S11 Strict zero amount

S12 unknown policy
S13 negative amount
S14 unsupported currency
S15 missing time

Do not omit edge cases.

9. Validate pattern fidelity.

For each language explicitly identify:

- Context
- Strategy interface/protocol
- each ConcreteStrategy
- Client

Explain briefly why the implementation is Strategy and not merely conditional logic.

10. Compile/run actual implementations.

Check and report actual:

Java:
java version
javac version
compile command
test command

Python:
python3 version
test command

JavaScript:
node version
test command

C++:
g++ version
compile command
test command

Do not claim anything ran unless it actually ran.

11. Record actual test results.

For each language provide:

passed / total
failures
errors
exit code

All 15 shared cases must pass before calling that language complete.

If a language fails, fix the implementation and rerun it.

Do not change the shared expected results to make a failing implementation pass.

12. Measure source LOC.

For each language:

- count production source only
- exclude tests
- exclude blank lines
- exclude comment-only lines
- exclude fixture files
- exclude generated/compiler output

Record the exact count.

13. Measure maximum function cyclomatic complexity.

Use the previously frozen McCabe-style rule:

start at 1
add for:
- conditional branches
- loops
- switch cases
- exception handlers
- ternary decisions
- short-circuit Boolean decisions

Apply the same measurement method consistently.

Record:

max_function_cc

for each implementation.

14. Measure process runtime.

For each language:

- compile first where required
- perform one warm-up
- then perform five measured fresh-process runs
- each measured run must process the complete Strategy fixture set
- record all five raw times
- report the median in milliseconds

Do not estimate missing timings.

15. Create evidence files inside each language folder.

Each language folder should contain:

production source
test source/runner
README.md
test_result.txt
timings.csv

README must include:

- selected source pattern ID S3-N01
- instance name Cancellation Policy Evaluation
- Strategy role mapping
- compile/run instructions
- simplifications from Rails
- statement that no real booking cancellation occurs

16. Do not create the final cross_language.csv yet.

Instead prepare four append-ready records containing:

pattern_name
instance_name
language
snippet_path
source_loc
max_function_cc
runtime_compiler
test_command
test_result
median_process_ms
language_notes

Save them in a temporary machine-readable artifact for later aggregation.

17. Package this stage.

Create one ZIP containing:

Part3/common_specs/strategy_cases.tsv
Part3/common_specs/strategy_expected.tsv
Part3/strategy/java/
Part3/strategy/python/
Part3/strategy/javascript/
Part3/strategy/cpp/
Strategy append-ready records
verification summary

18. Do not modify DaPlanStan.
19. Do not implement any other pattern yet.

At the end give me:

1. Exact files created.
2. Shared fixture validation.
3. Java implementation structure and result.
4. Python implementation structure and result.
5. JavaScript implementation structure and result.
6. C++ implementation structure and result.
7. Pattern-role mapping in all four languages.
8. Test results.
9. Runtime/compiler versions.
10. LOC values.
11. Maximum cyclomatic complexity values.
12. Five raw timing measurements and median for each language.
13. Append-ready cross-language records for Strategy.
14. Remaining limitations.
15. Downloadable Strategy cross-language ZIP.

Do not begin Observer yet.

Worked for 7m 44s

DaPlanStan — Part 3: Strategy Cross-Language Implementation

Completed — 60/60 tests passed

Selected pattern: S3-N01 — Strategy (GoF)
Confirmed instance: Cancellation Policy Evaluation

The Strategy collaboration has been implemented and tested in Java, Python, JavaScript and C++. All four implementations use the same frozen S01–S15 fixtures and expected results.

Implementations completed	Shared tests passed
4/4	**60/60**
All four languages	15 per language · 0 failures

1. Download the implementation

Complete Strategy Cross-Language Package

34 files · ZIP integrity verified · Source, tests, fixtures and measurements

Download Strategy S3-N01 ZIP ↗

Verification Summary ↗

Four Append-Ready Records ↗

Shared Strategy Cases ↗ · Expected Results ↗

The ZIP excludes compiled binaries and temporary Python caches. Java and C++ compilation commands are documented so the project can be reproduced on another machine.

2. Exact files created

Plain text

```
Part3/
├── STRATEGY_VERIFICATION_SUMMARY.md
├── common_specs/
│   ├── README.md
│   ├── strategy_cases.tsv
│   └── strategy_expected.tsv
├── strategy/
```

```
│   ├── strategy_append_ready.json
│   ├── java/
│   │   ├── Main.java
│   │   ├── TestMain.java
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── python/
│   │   ├── main.py
│   │   ├── test_main.py
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── javascript/
│   │   ├── main.js
│   │   ├── test_main.js
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   └── cpp/
│       ├── main.hpp
│       ├── main.cpp
│       ├── test_main.cpp
│       ├── README.md
│       ├── test_result.txt
│       └── timings.csv
```

3. Shared fixture validation

All implementations use the frozen 15 test cases.

Cases	Coverage
S01–S04	Moderate policy at 72h and 24h boundaries
S05–S08	Flexible policy near and at start
S09	Strict always charges 100%

S10	€0.05 half-up rounding: €0.03 fee, €0.02 refund
S11	Zero amount
S12	Unknown policy
S13	Negative amount
S14	Unsupported currency
S15	Missing start-time value

The tests produced the expected deterministic error codes: `UNKNOWN_POLICY`, `INVALID_AMOUNT`, `UNSUPPORTED_CURRENCY` and `INVALID_TIME`.

The fixture SHA-256 hashes are recorded in `common_specs/README.md` and the verification summary. No expected results were modified to obtain passing tests.

4. Pattern-role mapping in all four languages

Strategy role	Java	Python	JavaScript	C++
Context	`QuoteEvaluator`	`QuoteEvaluator`	`QuoteEvaluator`	`QuoteEvaluator`
Strategy interface	`FeeStrategy` interface	`FeeStrategy` ABC	Shared `feeRate` protocol	Abstract `FeeStrategy`
ConcreteStrategy 1	`FlexiblePolicy`	`FlexiblePolicy`	`FlexiblePolicy`	`FlexiblePolicy`
ConcreteStrategy 2	`ModeratePolicy`	`ModeratePolicy`	`ModeratePolicy`	`ModeratePolicy`
ConcreteStrategy 3	`StrictPolicy`	`StrictPolicy`	`StrictPolicy`	`StrictPolicy`
Client	`TestMain`	`test_main`	`test_main`	`test_main`

Why this is Strategy: The `QuoteEvaluator` chooses an approved policy and delegates the varying fee-rate algorithm to a separate policy object. The three algorithms are not collapsed into a single conditional calculation.

All four versions preserve this collaboration despite using different language mechanisms for interfaces and polymorphism.

5. Actual compilation and test results

Language	Actual runtime/compiler	Test command (within language folder)	Passed	Failures	Errors	Exit
Java	OpenJDK / javac 21.0.12.1	`java TestMain`	15/15	0	0	0
Python	Python 3.13.5	`python3 test_main.py`	15/15	0	0	0
JavaScript	Node.js v22.16.0	`node test_main.js`	15/15	0	0	0
C++	g++ 14.2.0, C++17	`./strategy_tests`	15/15	0	0	0

Compilation succeeded using:

Java:

```bash
javac Main.java TestMain.java
java TestMain
```

C++:

```bash
g++ -std=c++17 -O2 -Wall -Wextra -pedantic main.cpp test_main.cpp -o strategy_tests
./strategy_tests
```

Every test runner read both shared TSV files and compared generated output against the expected file.

6. Source LOC and cyclomatic complexity

Language	Production LOC	Maximum function CC

Java	52	12
Python	54	8
JavaScript	42	8
C++	74	15

Measurements exclude tests, fixtures, blank lines and comment-only lines. C++ production LOC includes both `main.hpp` and `main.cpp`.

The reproducible measurement script applies the agreed McCabe-style counting rules: starting at 1, adding decision points for branches, loops, exception handlers, ternaries and short-circuit operators. These are source-level measurements, with the lexical-analysis limitations documented in the verification summary.

7. Actual runtime measurements

The measurements used one warm-up followed by five measured fresh-process executions for each language.

All values are elapsed milliseconds. The median excludes the warm-up and includes process startup, fixture loading, evaluation, result checking and output.

Language	Run 1	Run 2	Run 3	Run 4	Run 5	Median
Java	77.186	91.907	104.042	118.962	87.222	**91.907**
Python	846.949	837.543	827.757	847.219	839.469	**839.469**
JavaScript	30.074	38.741	30.426	33.440	29.249	**30.426**
C++	2.665	2.559	2.738	2.241	2.582	**2.582**

Median fresh-process execution time (ms)

These measurements should **not** be interpreted as pure policy-algorithm performance. For this small workload, runtime startup and environment overhead can dominate the elapsed time.

The raw timings, including the warm-up measurements, are preserved in each language's `timings.csv`.

8. Append-ready Strategy records

Four machine-readable records are stored in:

Part3/strategy/strategy_append_ready.json ↗

Each record includes the exact eleven faculty fields:

`pattern_name`, `instance_name`, `language`, `snippet_path`, `source_loc`, `max_function_cc`, `runtime_compiler`, `test_command`, `test_result`, `median_process_ms`, `language_notes`.

All four use:

- `pattern_name`: `Strategy`
- `instance_name`: `Cancellation Policy Evaluation`

The records are ready for incorporation into the final dataset after the remaining four pattern types are implemented. The final `cross_language.csv` has **not** been created.

9. Remaining limitations

This implementation is a standalone representation of the verified S3-N01 collaboration. It deliberately does not reproduce Rails authorization, email-linked booking eligibility checks, Active Record persistence or live booking integrations.

The tests cover the fixed 15 shared cases and their expected output. They do not establish behavior for every possible integer or malformed input. Timing results are specific to the current execution environment.

No real booking cancellation, payment, refund, email or external API request occurs.

10. Final status

- ✓ Frozen S01–S15 fixtures created
- ✓ Java, Python, JavaScript and C++ implementations completed
- ✓ All 60 language-specific tests passed
- ✓ Runtime/compiler versions recorded
- ✓ LOC, complexity and five process timings measured per language
- ✓ Four append-ready records and ZIP created
- ✓ DaPlanStan and frozen evolution records remain unchanged

S3-N01 — Strategy (GoF) is complete for the cross-language coursework stage.

Only Strategy was implemented. Controller, Information Expert, Observer and Adapter have not been started.

strategy_append_ready.json	Code
strategy_cases.tsv	Spreadsheet
strategy_expected.tsv	Spreadsheet

Now continue the cross-language implementation phase.

Strategy S3-N01 is complete and frozen.

Implement ONLY the second selected pattern:

P08 — Observer (GoF)
Outbound Mail Delivery Logging

Do not modify the completed Strategy files.

Do not implement Controller, Information Expert or Adapter yet.

Use the frozen Observer specification exactly as previously defined.

1. Extend the existing Part3 working structure.

Preserve:

Part3/common_specs/strategy_cases.tsv
Part3/common_specs/strategy_expected.tsv
Part3/strategy/
all Strategy evidence and measurements

Create the Observer fixture files:

Part3/common_specs/observer_cases.tsv
Part3/common_specs/observer_expected.tsv

Use the frozen cases O01–O07 exactly.

2. The simplified problem is:

A MailDeliveryPublisher simulates message delivery.

A MailLogger may register as an observer.

When delivery succeeds, the publisher notifies all currently registered observers through the observer callback.

Nothing is actually emailed.

The publisher must not contain special-case code that directly calls MailLogger.

3. Preserve these Observer roles in all four languages:

Subject / Publisher:
MailDeliveryPublisher

Observer contract:
onDelivered(message)

ConcreteObserver:
MailLogger

Event:
MailMessage

Registration:
register(observer)
unregister(observer)

Client:
test runner

The publisher must notify through the registered observer interface/protocol.

4. Frozen input fields:

case_id
register_logger
delivered
message_id
subject
recipient

Values:

register_logger:
YES or NO

delivered:
YES or NO

Each normal case creates a fresh publisher and logger unless the case explicitly defines a sequence.

5. Frozen output rules.

If a registered logger receives a successful delivery event:

case_id LOGGED message_id subject recipient

Otherwise:

case_id NO_NOTIFICATION

Controlled errors must use deterministic error codes.

6. Implement the exact frozen cases:

O01:
registered + delivered
→ LOGGED with correct message fields

O02:
not registered + delivered
→ NO_NOTIFICATION

O03:
registered + delivery failed
→ NO_NOTIFICATION

O04:
registered + another valid delivered message
→ LOGGED with that message's fields

O05:
delivery request with missing recipient
→ ERROR INVALID_RECIPIENT

O06:
invalid registration flag
→ ERROR INVALID_FLAG

O07:
register logger
→ unregister logger

→ perform successful delivery
→ NO_NOTIFICATION

Do not change these expected outcomes.

7. Create exactly four language implementations under:

Part3/observer/java/
Part3/observer/python/
Part3/observer/javascript/
Part3/observer/cpp/

Do not use Rails, Action Mailer, network APIs or external libraries.

8. Java implementation.

Keep explicitly identifiable:

Observer interface
MailLogger
MailMessage
MailDeliveryPublisher

MailDeliveryPublisher must store registered observers and notify them through the interface.

Use simple dependency-free Java.

9. Python implementation.

Preserve separate publisher and observer responsibilities.

Do not replace Observer with one direct logger call.

An ABC/protocol-style class or clear duck-typed observer contract is acceptable.

Use no third-party packages.

10. JavaScript implementation.

Use Node.js only.

Publisher must maintain its observer collection and invoke a common callback.

Do not hard-code MailLogger into the publisher.

11. C++ implementation.

Use standard C++ only.

Represent the Observer contract explicitly, such as an abstract base interface.

Publisher must hold observer references/pointers safely for this small test lifetime.

Do not use external libraries.

12. All four test runners must read:

Part3/common_specs/observer_cases.tsv

and compare actual results against:

Part3/common_specs/observer_expected.tsv

Do not hard-code separate expected outputs into the language implementations.

13. Test registration behavior explicitly.

Verification must establish:

- registered observer receives successful delivery
- unregistered observer receives nothing
- failed delivery produces no notification
- unregister removes notification eligibility
- invalid input produces controlled errors
- message fields received by observer match the delivered event

14. Pattern-fidelity check.

For every language identify:

Subject
Observer interface/protocol
ConcreteObserver
Event object
register method
unregister method
notification point

Explain why the implementation is Observer rather than ordinary direct logging.

A design where sending code directly calls:

MailLogger.log(...)

must be rejected.

15. Compile and execute actual implementations.

Record:

Java:
java version
javac version
compile command
test command

Python:
python3 version
test command

JavaScript:
node version
test command

C++:
g++ version
compile command
test command

Do not claim success without execution.

16. All seven cases must pass in every language.

Expected total:

7 Java
7 Python
7 JavaScript
7 C++

28 total Observer tests.

If a test fails, fix the implementation and rerun it.

Do not alter the frozen fixtures merely to obtain a passing result.

17. Measure production LOC.

Apply exactly the same rule used for Strategy:

- production source only
- exclude tests
- exclude fixtures
- exclude blank lines
- exclude comment-only lines
- exclude generated binaries/cache files

Record source_loc for each language.

18. Measure maximum function cyclomatic complexity.

Use the same frozen McCabe-style method used for Strategy.

Record max_function_cc for each language.

Do not change the measurement rule between patterns.

19. Measure execution time.

For each language:

- compile first where applicable
- one warm-up process run
- five measured fresh-process runs
- each run processes the complete O01–O07 fixture set
- record all five elapsed times
- report median_process_ms

Use the same timing procedure as Strategy.

Do not compare the languages as pure algorithm benchmarks; process startup may dominate.

20. Create evidence files in each language folder:

production source
test runner
README.md
test_result.txt
timings.csv

README must state:

source ID P08
pattern Observer (GoF)
instance Outbound Mail Delivery Logging
role mapping

compile/run commands
difference from Rails/Action Mailer
no real email is sent

21. Prepare four append-ready records.

Use the exact faculty fields:

pattern_name
instance_name
language
snippet_path
source_loc
max_function_cc
runtime_compiler
test_command
test_result
median_process_ms
language_notes

Use:

pattern_name = Observer
instance_name = Outbound Mail Delivery Logging

Save as:

Part3/observer/observer_append_ready.json

Do not create the final cross_language.csv yet.

22. Create an Observer verification summary:

Part3/OBSERVER_VERIFICATION_SUMMARY.md

Include:

fixture validation
role mapping
compilation
tests
LOC
complexity
raw timings
medians
limitations

23. Package this stage.

Create one ZIP containing the cumulative Part3 materials required for Observer verification, including:

Observer fixtures
Observer implementations
Observer append-ready data
Observer verification summary

It may include the already frozen common specification structure, but do not modify completed Strategy source or Strategy results.

Exclude compiled binaries and temporary cache files.

24. Do not modify DaPlanStan.

25. Do not begin Controller, Information Expert or Adapter.

At the end give me:

1. Exact Observer files created.
2. Fixture validation.
3. Java result.
4. Python result.
5. JavaScript result.
6. C++ result.
7. Observer role mapping.
8. Test results.
9. Runtime/compiler versions.
10. LOC values.
11. Maximum function CC values.
12. Five raw timing measurements and median for each language.
13. Four append-ready Observer records.
14. Remaining limitations.
15. Downloadable Observer cross-language ZIP.

Do not begin Controller yet.

Worked for 4m 44s

Part 3 — Observer Cross-Language Implementation

Completed — 28/28 tests passed

Pattern: Observer (GoF)
Source ID: P08
Confirmed instance: Outbound Mail Delivery Logging

The Observer pattern has been implemented and tested in Java, Python, JavaScript and C++ using the same frozen O01–O07 fixtures.

Strategy S3-N01 remains unchanged. No DaPlanStan application code or other pattern implementation was modified.

1. Download the completed Observer package

Observer P08 Cross-Language Package

ZIP · 64 KB · 68 files including preserved Strategy materials

Download Observer Cross-Language ZIP ↗

Observer Verification Summary ↗

Four Append-Ready Records ↗

Shared Observer Cases ↗ · Expected Results ↗

The ZIP passed integrity verification and excludes compiled binaries and temporary caches.

2. Exact files created

Plain text

```
Part3/
├── OBSERVER_VERIFICATION_SUMMARY.md
├── common_specs/
│   ├── OBSERVER_README.md
│   ├── observer_cases.tsv
│   └── observer_expected.tsv
├── observer/
│   ├── observer_append_ready.json
│   ├── java/
│   │   ├── Main.java
│   │   ├── TestMain.java
│   │   ├── README.md
```

```
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── python/
│   │   ├── main.py
│   │   ├── test_main.py
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── javascript/
│   │   ├── main.js
│   │   ├── test_main.js
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   └── cpp/
│       ├── main.hpp
│       ├── main.cpp
│       ├── test_main.cpp
│       ├── README.md
│       ├── test_result.txt
│       └── timings.csv
```

3. Shared fixture verification

Both common files were created and frozen before implementation.

Case	Test condition	Actual result in all four languages
O01	Registered observer, successful delivery	`LOGGED`
O02	No registered observer	`NO_NOTIFICATION`
O03	Registered observer, failed delivery	`NO_NOTIFICATION`
O04	Different successfully delivered message	`LOGGED` , correct message fields
O05	Missing recipient	`ERROR INVALID_RECIPIENT`
O06	Invalid registration flag	`ERROR INVALID_FLAG`

O07	Register → unregister → deliver	`NO_NOTIFICATION`

All four runners read `observer_cases.tsv` and compare output line by line against `observer_expected.tsv`. Expected answers are not hard-coded separately in each language.

The files' SHA-256 checksums are recorded in `common_specs/OBSERVER_README.md`.

4. Observer roles in all four languages

Role	Java	Python	JavaScript	C++
Subject	`MailDeliveryPublisher`	`MailDeliveryPublisher`	`MailDeliveryPublisher`	`MailDeliveryPublisher`
Observer contract	`MailObserver` interface	`MailObserver` ABC	`MailObserver` base contract	Abstract `MailObserver`
ConcreteObserver	`MailLogger`	`MailLogger`	`MailLogger`	`MailLogger`
Event	`MailMessage`	`MailMessage` dataclass	`MailMessage`	`MailMessage` struct
Registration	`register / unregister`	`register / unregister`	`register / unregister`	`registerObserver / unregisterObserver`
Notification point	`deliver`	`deliver`	`deliver`	`deliver`

In C++, `registerObserver` is used because `register` is a C++ keyword.

Why this is Observer: Each publisher maintains registered observers and notifies them through the common `onDelivered(message)` callback after successful delivery. The publisher contains no special-case direct logging call to `MailLogger`.

5. Compilation and actual test results

Language	Runtime/compiler	Test command	Passed	Failures	Errors	Exit
Java	OpenJDK / javac 21.0.12.1	`java TestMain`	7/7	0	0	0

Python	Python 3.13.5	`python3 test_main.py`	7/7	0	0	0
JavaScript	Node.js v22.16.0	`node test_main.js`	7/7	0	0	0
C++	g++ 14.2.0, C++17	`./observer_tests`	7/7	0	0	0

Java compilation succeeded with:

```bash
javac Main.java TestMain.java
java TestMain
```

C++ compilation succeeded with:

```bash
g++ -std=c++17 -O2 -Wall -Wextra -pedantic main.cpp test_main.cpp -o observer_tests
./observer_tests
```

Total: 28 passed, 0 failures, 0 errors.

6. Production LOC and cyclomatic complexity

Language	Production LOC	Maximum function CC
Java	47	5
Python	32	5
JavaScript	43	5
C++	60	4

LOC excludes tests, fixtures, blank lines, comment-only lines and compiled output. The C++ figure includes both `main.hpp` and `main.cpp`.

The measurements reuse Strategy's McCabe-style counting rules. The Observer-specific measurement script adapts method detection to Observer source structures without changing the counting method.

These are reproducible static measurements, subject to the lexical-analysis limitations described in the verification summary.

7. Five raw timing measurements and median

Each language was run once for warm-up, followed by five measured fresh-process executions. Every run processed all seven cases successfully.

All times in milliseconds; the median is calculated from the five measured runs, excluding warm-up.

Language	Run 1	Run 2	Run 3	Run 4	Run 5	Median
Java	126.393	104.699	103.542	116.815	109.548	**109.548**
Python	1158.487	1127.801	1101.022	1133.245	1250.233	**1133.245**
JavaScript	36.193	46.431	44.342	43.673	37.155	**43.673**
C++	2.168	2.149	2.421	2.325	2.395	**2.325**

Median fresh-process execution time (ms)

The measurements include process startup, fixture reading, Observer execution, expected-output comparison and output writing. They should not be interpreted as isolated algorithm performance.

The raw timing evidence, including warm-up results, is saved in each language's `timings.csv`.

8. Append-ready Observer records

The file observer_append_ready.json ↗ contains four completed records using the exact faculty fields:

`pattern_name`, `instance_name`, `language`, `snippet_path`, `source_loc`, `max_function_cc`, `runtime_compiler`, `test_command`, `test_result`, `median_process_ms`, `language_notes`.

Every record uses:

- `pattern_name = Observer`
- `instance_name = Outbound Mail Delivery Logging`

The records are ready for later aggregation, but the final `cross_language.csv` has not been generated.

9. Remaining limitations

This is a dependency-free simulation of the source-confirmed P08 collaboration, not a Rails or Action Mailer integration test. No real email is sent.

The seven frozen cases test registration, nonregistration, failed delivery, unregistering, message-field correctness and controlled errors. They do not cover concurrency, persistent logging or every possible invalid message.

C++ stores nonowning observer pointers for this deliberately short test lifetime. The test objects remain valid throughout notification.

10. Final status

- Frozen Observer fixtures created and verified
- Four language implementations completed
- All 28 tests passed
- LOC, complexity, compiler versions and timing evidence recorded
- Four append-ready records packaged
- Frozen Strategy files and results preserved by checksum

P08 — Observer (GoF) is complete for the cross-language stage.

Strategy and Observer are now complete. Controller, Information Expert and Adapter have not been implemented, and no DaPlanStan source was changed.

- **observer_append_ready.json**
  Code
- **observer_cases.tsv**
  Spreadsheet
- **observer_expected.tsv**
  Spreadsheet

Now continue the cross-language implementation phase.

Strategy S3-N01 and Observer P08 are complete and frozen.

Implement ONLY the third selected pattern:

P03 — Controller (GRASP)
Trip Creation Coordination

Do not modify completed Strategy or Observer files.

Do not implement Information Expert or Adapter yet.

Use the frozen Controller common specification exactly as previously defined.

1. Extend the existing Part3 structure.

Preserve all existing:

Part3/common_specs/strategy_*
Part3/common_specs/observer_*
Part3/strategy/
Part3/observer/
existing evidence and append-ready records

Create:

Part3/common_specs/controller_cases.tsv
Part3/common_specs/controller_expected.tsv

Freeze the exact cases C01–C08 before implementation.

2. Simplified problem.

A user submits a request to create a trip.

TripCreationController receives that system event and coordinates:

- Trip creation
- initial owner membership creation
- in-memory persistence
- simulated cover-image task scheduling

There is:

- no database
- no Rails
- no image API
- no network call

The Controller must coordinate collaborators rather than perform every responsibility itself.

3. Preserve these roles in all four languages.

GRASP Controller:
TripCreationController

Domain entity:
Trip

Account/owner context:
Account

Membership entity:
TripMember

Persistence collaborator:
TripStore

Asynchronous collaborator:
CoverTaskQueue

Client:
test runner / request handler

The test runner must send the request to TripCreationController rather than manually orchestrating all collaborators itself.

4. Frozen input columns.

Use:

case_id
user_id
owner_traveler_id
title
start_date
end_date

Use literal:

~

for absent optional/input values.

Dates use:

YYYY-MM-DD

5. Frozen validation rules.

A request succeeds only if:

- user_id is present
- owner_traveler_id is present

- title is nonblank after trimming
- start_date is a valid Gregorian date
- end_date is a valid Gregorian date
- end_date is not earlier than start_date

Same-day trips are valid.

Do not accept impossible dates such as:

2026-02-30

6. Success behavior.

On success create exactly:

- one Trip
- one TripMember with role `owner`
- one cover-task request

The generated trip ID must be:

trip-<case_id>

Example:

C01 → trip-C01

Normalize the title by trimming leading and trailing whitespace.

7. Failure behavior.

On validation failure create:

- zero Trip records
- zero TripMembers
- zero cover tasks

The operation must therefore behave atomically at the simplified in-memory level.

Use these deterministic error codes:

UNAUTHENTICATED
MISSING_OWNER
MISSING_TITLE
INVALID_DATE
INVALID_DATE_RANGE

Do not expose language-specific exception text.

8. Freeze the exact C01–C08 cases.

C01:
valid trip
start = 2026-11-16
end = 2026-11-18
→ CREATED
→ one owner membership
→ cover queued

C02:
user_id = ~
otherwise valid

→ ERROR UNAUTHENTICATED

C03:
owner_traveler_id = ~
otherwise valid
→ ERROR MISSING_OWNER

C04:
title contains only spaces
→ ERROR MISSING_TITLE

C05:
start_date = 2026-02-30
→ ERROR INVALID_DATE

C06:
valid dates but end date earlier than start date
→ ERROR INVALID_DATE_RANGE

C07:
same valid start and end date
→ CREATED

C08:
title exactly:

  City Break

including surrounding spaces in the fixture
→ CREATED
→ normalized title = City Break

Use deterministic valid user and traveler identifiers for the successful/otherwise-valid fields.

9. Output format.

Success output must have exactly:

case_id
CREATED
trip_id
normalized_title
owner_traveler_id
owner
COVER_QUEUED

Failure output must have exactly:

case_id
ERROR
error_code

Use tab-separated deterministic output.

10. Implement Java version under:

Part3/controller/java/

Keep clearly identifiable:

TripCreationController
Trip

Account
TripMember
TripStore
CoverTaskQueue

Use dependency-free Java.

Use the standard Java date library for strict Gregorian validation.

Do not hide all domain behavior inside TestMain.

11. Implement Python version under:

Part3/controller/python/

Preserve the same collaborators and Controller responsibility.

Use Python standard library only.

Use strict date parsing/validation.

Do not implement the entire use case as one standalone procedural function.

12. Implement JavaScript version under:

Part3/controller/javascript/

Use Node.js only.

Preserve the same participant responsibilities.

Be careful: JavaScript Date normalization must not silently accept impossible dates such as 2026-02-30.

Implement strict validation so C05 is rejected.

13. Implement C++ version under:

Part3/controller/cpp/

Use standard C++ only.

Preserve clear domain collaborators and controller coordination.

Implement strict Gregorian date validation without external libraries.

14. All four test runners must read:

Part3/common_specs/controller_cases.tsv

and compare actual output against:

Part3/common_specs/controller_expected.tsv

Do not hard-code independent expected answers per language.

15. Test side effects as well as output.

For every successful case verify:

TripStore contains exactly one newly created Trip for the request.

One owner TripMember was recorded.

One cover task was queued for that Trip.

For every failed case verify:

TripStore unchanged.

No membership created.

No cover task queued.

16. Validate GRASP Controller fidelity.

For every language identify:

System event:
create-trip request

Controller:
TripCreationController

Collaborators:
Trip
Account
TripMember
TripStore
CoverTaskQueue

Outcome:
created trip + owner membership + queued task

Explain why this demonstrates GRASP Controller.

Do not claim that simply naming a class Controller establishes the pattern.

The important evidence is that it receives the system event and coordinates the relevant domain collaborators.

17. Compile and execute all four implementations.

Record actual:

Java:
java version
javac version
compile command
test command

Python:
python3 version
test command

JavaScript:
node version
test command

C++:
g++ version
compile command
test command

Do not claim execution unless it actually occurred.

18. Required test result:

8 Java cases
8 Python cases
8 JavaScript cases
8 C++ cases

Expected total:

32/32

If anything fails, fix the implementation and rerun.

Do not modify frozen expected results merely to make the implementation pass.

19. Measure production LOC using the same rules already used for Strategy and Observer:

- production code only
- exclude tests
- exclude fixtures
- exclude blank lines
- exclude comment-only lines
- exclude compiler/generated artifacts

Record exact source_loc per language.

20. Measure maximum function cyclomatic complexity.

Use the same previously frozen McCabe-style rule.

Do not change the complexity definition between patterns.

Record:

max_function_cc

for each language.

21. Measure process execution time consistently.

For each language:

- compile first where applicable
- one warm-up
- five measured fresh-process runs
- each run processes C01–C08 completely
- record each elapsed time
- calculate median_process_ms from the five measured runs

Retain raw timing evidence.

Do not interpret these as pure algorithm benchmarks because startup and fixture parsing may dominate.

22. Create evidence files inside every language folder:

production source
test runner
README.md
test_result.txt
timings.csv

README must include:

- source ID P03
- pattern Controller (GRASP)
- instance name Trip Creation Coordination
- participant mapping
- compile/run commands
- simplification from Rails
- statement that no real database or external cover-image API is used

23. Prepare four append-ready faculty records using exactly:

pattern_name
instance_name
language
snippet_path
source_loc
max_function_cc
runtime_compiler
test_command
test_result
median_process_ms
language_notes

Use:

pattern_name = Controller

instance_name = Trip Creation Coordination

Save:

Part3/controller/controller_append_ready.json

Do not generate final cross_language.csv yet.

24. Create:

Part3/CONTROLLER_VERIFICATION_SUMMARY.md

Include:

- shared fixture validation
- participant mapping
- validation behavior
- atomic failure behavior
- compilation results
- test results
- LOC
- complexity
- five measured timings and medians
- limitations

25. Package this stage.

Create a cumulative ZIP containing the current Part3 materials needed to reproduce:

- frozen Strategy
- frozen Observer
- new Controller fixtures
- Controller implementations
- Controller append-ready records
- Controller verification evidence

Do not alter Strategy or Observer results.

Exclude:

- compiled Java .class files
- C++ binaries
- Python caches
- temporary files

Verify ZIP integrity.

26. Do not modify DaPlanStan.

27. Do not begin Information Expert or Adapter.

At the end give me:

1. Exact Controller files created.
2. C01–C08 fixture validation.
3. Java structure and test result.
4. Python structure and test result.
5. JavaScript structure and test result.
6. C++ structure and test result.
7. GRASP Controller participant mapping.
8. Side-effect/atomic-failure verification.
9. Runtime/compiler versions.
10. LOC values.
11. Maximum function CC values.
12. Five raw timing measurements and median for each language.
13. Four append-ready Controller records.

14. Remaining limitations.
15. Downloadable Controller cross-language ZIP.

Do not begin Information Expert yet.

Worked for 4m 52s

Part 3 — Controller (GRASP) Cross-Language Implementation

Completed — 32/32 tests passed

Pattern ID: P03
Pattern: Controller (GRASP)
Confirmed instance: Trip Creation Coordination

The Controller collaboration has been implemented and executed in Java, Python, JavaScript and C++. All four implementations use the same frozen C01–C08 fixtures.

Languages completed	Shared tests passed
4/4	**32/32**
Java, Python, JS, C++	Zero failures or errors

1. Download the completed Controller package

Controller P03 Cross-Language Package
Cumulative Part3 ZIP · 104 files · 103 KB

Download Controller P03 ZIP ↗

Controller Verification Summary ↗

Four Append-Ready Records ↗

Shared Controller Cases ↗ · Expected Results ↗

The ZIP passed integrity and source-manifest checks. All 48 previously frozen Strategy and Observer files passed checksum verification.

2. Exact Controller files created

Plain text

```
Part3/
├── CONTROLLER_VERIFICATION_SUMMARY.md
├── common_specs/
│   ├── CONTROLLER_README.md
│   ├── controller_cases.tsv
│   └── controller_expected.tsv
├── controller/
│   ├── controller_append_ready.json
│   ├── java/
│   │   ├── Main.java
│   │   ├── TestMain.java
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── python/
│   │   ├── main.py
│   │   ├── test_main.py
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── javascript/
│   │   ├── main.js
│   │   ├── test_main.js
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   └── cpp/
│       ├── main.hpp
│       ├── main.cpp
│       ├── test_main.cpp
│       ├── README.md
│       ├── test_result.txt
```

```
|       └── timings.csv
```

The cumulative package also includes the unchanged Strategy and Observer source, fixtures, measurements and append-ready records.

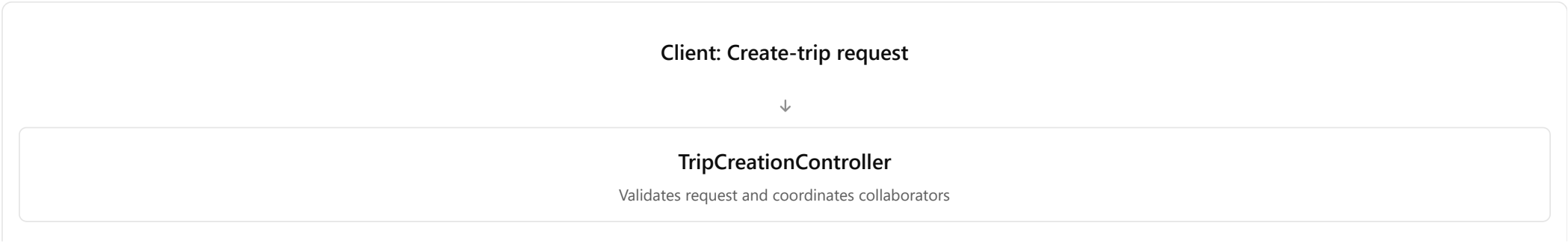
3. C01–C08 fixture validation

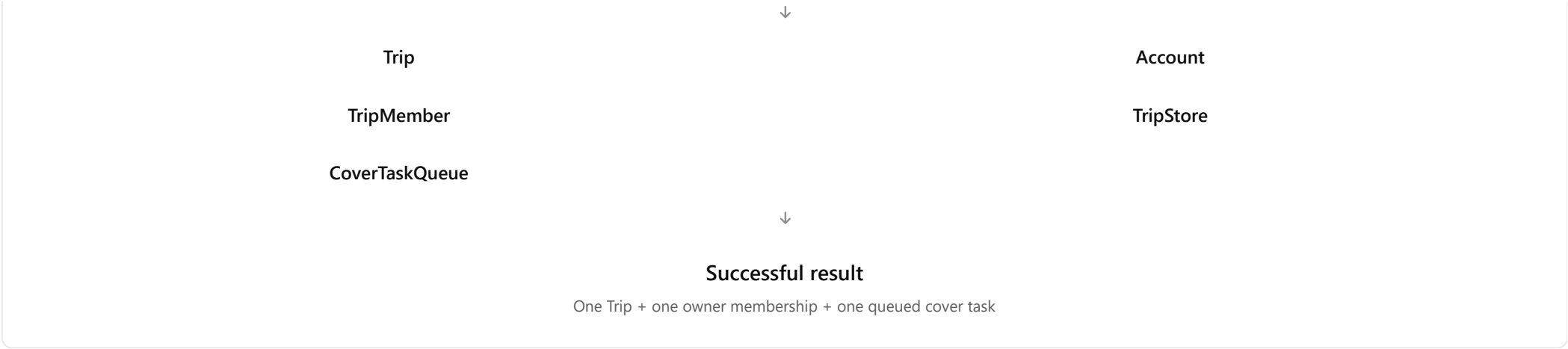
Both common fixtures were created and checksummed before implementation.

Case	Condition	Verified outcome
C01	Valid trip, 16–18 November 2026	`CREATED` , owner assigned, cover queued
C02	Missing user	`ERROR UNAUTHENTICATED`
C03	Missing owner traveler	`ERROR MISSING_OWNER`
C04	Whitespace-only title	`ERROR MISSING_TITLE`
C05	Impossible date `2026-02-30`	`ERROR INVALID_DATE`
C06	End date earlier than start	`ERROR INVALID_DATE_RANGE`
C07	Same-day trip	`CREATED`
C08	Title with surrounding spaces	`CREATED` , normalized to `City Break`

All four runners read the same TSV files and compared their actual output against `controller_expected.tsv` .

4. GRASP Controller collaboration





All four implementations retain these identifiable participants.

Role	Participant
System event	Create-trip request
GRASP Controller	`TripCreationController`
Domain entity	`Trip`
Account/owner context	`Account`
Membership entity	`TripMember`
In-memory persistence	`TripStore`
Simulated asynchronous work	`CoverTaskQueue`
Client	Fixture-driven test runner

This demonstrates GRASP Controller because the request handler delegates domain responsibilities rather than performing all operations in the test runner.

5. Atomic failure and side-effect verification

The test runners inspect the in-memory collaborators for every case.

Condition	TripStore	Owner memberships	Cover tasks
C01 success	1	1	1
C02–C06 validation errors	0	0	0
C07 success	1	1	1
C08 success	1	1	1

All validation occurs before creating side effects. The tests confirm that failed requests do not partially create trips, memberships or tasks.

This proves atomic behavior for the specified in-memory validation cases, not database transaction failure recovery.

6. Compilation and actual execution results

Language	Verified runtime/compiler	Test command	Passed	Failures	Errors	Exit
Java	OpenJDK / javac 21.0.12.1	`java TestMain`	8/8	0	0	0
Python	Python 3.13.5	`python3 test_main.py`	8/8	0	0	0
JavaScript	Node.js v22.16.0	`node test_main.js`	8/8	0	0	0
C++	g++ 14.2.0, C++17	`./controller_tests`	8/8	0	0	0

Java compilation:

```bash
javac Main.java TestMain.java
java TestMain
```

C++ compilation:

Bash

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic main.cpp test_main.cpp -o controller_tests
./controller_tests
```

All compilation and test commands executed successfully.

7. Production LOC and cyclomatic complexity

Language	Production LOC	Maximum function CC
Java	66	9
Python	59	10
JavaScript	55	9
C++	90	13

The LOC calculation excludes tests, fixtures, blank lines, comment-only lines and generated files.

Complexity uses the same McCabe-style counting helpers established for Strategy and reused for Observer. Only language-specific method discovery was adjusted for Controller.

8. Five raw timing measurements and medians

One warm-up preceded five measured fresh-process executions for each language. Every measured run successfully processed C01–C08.

Elapsed milliseconds; medians exclude warm-up.

Language	Run 1	Run 2	Run 3	Run 4	Run 5	Median
Java	127.034	160.614	159.473	462.171	100.443	**159.473**
Python	832.153	818.330	810.701	806.477	825.164	**818.330**
JavaScript	30.699	41.517	39.392	37.862	35.212	**37.862**

C++	2.464	2.566	2.789	3.347	2.068	**2.566**

Median fresh-process execution time (ms)

The measured process includes startup, fixture parsing, Controller execution, side-effect assertions and output comparison. These figures are not pure trip-creation algorithm benchmarks.

Raw six-run data, including each warm-up, is retained in the language folders.

9. Four append-ready faculty records

Download Controller Append-Ready Records ↗

The JSON contains four records with the eleven exact faculty fields:

`pattern_name`, `instance_name`, `language`, `snippet_path`, `source_loc`, `max_function_cc`, `runtime_compiler`, `test_command`, `test_result`, `median_process_ms`, `language_notes`.

Every record uses `pattern_name = Controller` and `instance_name = Trip Creation Coordination`.

The final `cross_language.csv` has not been generated.

10. Remaining limitations

The Controller example deliberately uses in-memory collaborators rather than Rails, Active Record or real background jobs. No database, image service or network request is involved.

Atomicity was tested for the frozen validation-failure scenarios, not exceptional failures in persistent storage or background-job scheduling. Runtime comparisons are machine-specific and influenced by startup overhead.

11. Final Part 3 progress

Pattern	Cross-language status
Strategy — S3-N01	Completed and frozen
Observer — P08	Completed and frozen
Controller — P03	**Completed and verified**
Information Expert — P01	Not started
Adapter — S2-N01	Not started

✓ **12 of the required 20 language implementations are now complete.**

The Controller ZIP is ready for the teammate handling Part 3 evidence and final aggregation.

No DaPlanStan application source was modified, and Information Expert or Adapter work has not begun.

controller_append_ready.json
Code

controller_cases.tsv
Spreadsheet

controller_expected.tsv
Spreadsheet

Pasted text(20261011-090846).txt
Document

Worked for 4m 50s

Part 3 — Information Expert (GRASP) Cross-Language Implementation

Completed — 32/32 tests passed

Pattern ID: P01
Pattern: Information Expert (GRASP)
Confirmed instance: Parsed Email to Trip Record Linking

The fourth selected pattern has been implemented and tested in Java, Python, JavaScript and C++ using the shared frozen I01–I08 fixtures.

Strategy, Observer and Controller remain unchanged. Adapter has not been started.

1. Download the completed package

Information Expert P01 Cross-Language Package
Cumulative Part3 ZIP · 137 files · 132 KB · Integrity verified

Download Information Expert P01 ZIP ↗

Verification Summary ↗

Four Append-Ready Records ↗

Shared I01–I08 Cases ↗ · Expected Results ↗

The archive contains all four completed pattern implementations and the frozen Strategy, Observer and Controller materials.

2. Exact Information Expert files created

Plain text

```
Part3/
├── INFORMATION_EXPERT_VERIFICATION_SUMMARY.md
├── common_specs/
│   ├── INFORMATION_EXPERT_README.md
│   ├── information_expert_cases.tsv
│   └── information_expert_expected.tsv
├── information_expert/
│   ├── information_expert_append_ready.json
│   ├── java/
│   │   ├── Main.java
│   │   ├── TestMain.java
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── python/
│   │   ├── main.py
│   │   ├── test_main.py
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── javascript/
│   │   ├── main.js
│   │   ├── test_main.js
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   └── cpp/
│       ├── main.hpp
│       ├── main.cpp
│       ├── test_main.cpp
│       ├── README.md
│       ├── test_result.txt
│       └── timings.csv
```

3. Frozen fixture validation

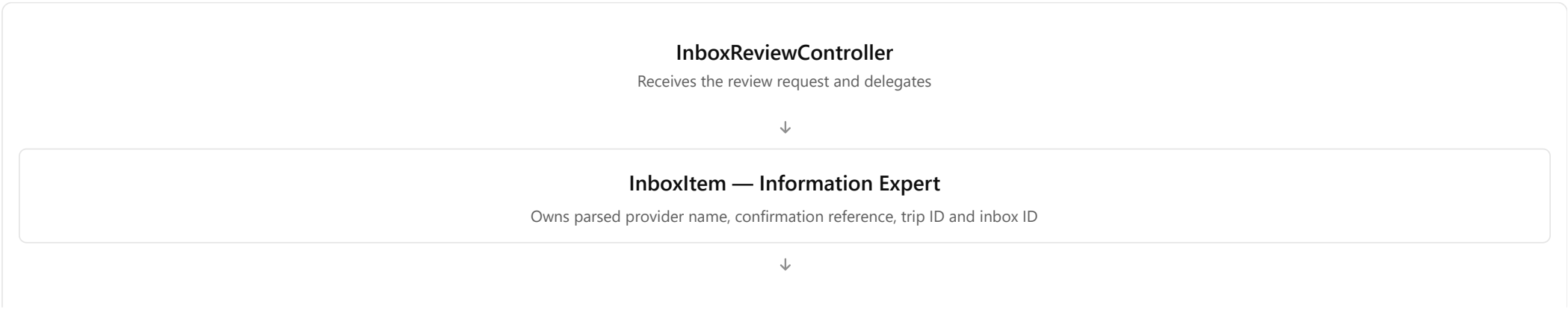
The fixtures were created and checksummed before implementing the translations. All four test runners use the same input and expected-output files.

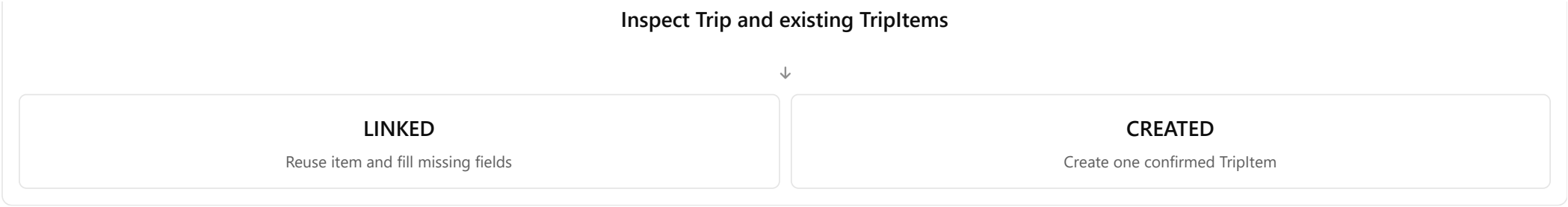
Case	Test scenario	Verified outcome
I01	Existing same-trip item has missing reference and inbox link	`LINKED` ; both fields filled
I02	No existing item	`CREATED` ; `item-I02`
I03	Existing item has a confirmation reference	`LINKED` ; original reference preserved
I04	Matching item belongs to another trip	`CREATED` in current trip
I05	Provider name missing	`ERROR MISSING_PROVIDER`
I06	Provider and item names differ only in letter case	`LINKED`
I07	Existing item already belongs to another InboxItem	`LINKED` ; original inbox link preserved
I08	Incoming confirmation reference absent	`LINKED` ; reference remains missing

All eight cases produce their exact expected TSV output.

The fixture SHA-256 values are recorded in `common_specs/INFORMATION_EXPERT_README.md` .

4. GRASP Information Expert collaboration





Role mapping in all four languages

Role	Java	Python	JavaScript	C++
Information Expert	`InboxItem`	`InboxItem`	`InboxItem`	`InboxItem`
Expert operation	`linkOrCreateTripItem`	`link_or_create_trip_item`	`linkOrCreateTripItem`	`linkOrCreateTripItem`
Caller	`InboxReviewController`	Same	Same	Same
Collection context	`Trip`	`Trip`	`Trip`	`Trip`
Domain record	`TripItem`	`TripItem`	`TripItem`	`TripItem`

Why this is Information Expert: The matching and linking decision belongs to `InboxItem`, which possesses the relevant parsed booking information.

The caller delegates that responsibility. The example does not depend on Rails controller syntax, transform unrelated provider schemas or introduce a separate factory abstraction.

5. State-change verification

The test runners check more than the final text output.

Requirement	Verified behavior
Existing matching item reused	No additional TripItem created

Missing confirmation reference	Filled from incoming information
Existing nonblank confirmation reference	Preserved
Missing inbox link	Filled
Existing inbox link	Preserved
Cross-trip item	Never reused
New item	Correct `item-Ixx` ID and trip ID
New item status	`confirmed`
Invalid provider name	No record created or modified

These checks passed in all four languages.

6. Compilation and actual test results

Language	Verified runtime/compiler	Test command	Passed	Failures	Errors	Exit
Java	OpenJDK / javac 21.0.12.1	`java TestMain`	8/8	0	0	0
Python	Python 3.13.5	`python3 test_main.py`	8/8	0	0	0
JavaScript	Node.js v22.16.0	`node test_main.js`	8/8	0	0	0
C++	g++ 14.2.0, C++17	`./information_expert_tests`	8/8	0	0	0

Java compilation and execution:

Bash

```
javac Main.java TestMain.java
java TestMain
```

C++ compilation and execution:

```bash
g++ -std=c++17 -O2 -Wall -Wextra -pedantic main.cpp test_main.cpp -o information_expert_tests
./information_expert_tests
```

Both compilation commands succeeded with exit code 0.

Total: 32/32 tests passed, zero failures and zero errors.

7. Production LOC and cyclomatic complexity

Language	Production LOC	Maximum function CC
Java	63	7
Python	45	7
JavaScript	38	6
C++	83	7

LOC excludes tests, fixtures, blank lines, comment-only lines and compiled files. C++ includes both `main.hpp` and `main.cpp`.

The complexity audit reused the frozen Part3 McCabe-style counting helpers. Only method-discovery patterns were extended for the new Information Expert classes.

These are source-level measurements, not profiler measurements.

8. Five raw timing measurements and medians

Each language completed one warm-up and five measured fresh-process executions. Every measured execution passed all eight cases.

All values are milliseconds. Medians exclude the warm-up.

Language	Run 1	Run 2	Run 3	Run 4	Run 5	Median
Java	75.602	64.794	78.340	78.457	56.485	**75.602**
Python	847.285	798.344	814.468	819.622	839.151	**819.622**
JavaScript	52.606	33.741	37.011	36.436	33.055	**36.436**
C++	4.391	4.827	4.058	2.721	4.930	**4.391**

Median fresh-process execution time

The timings include process startup, reading fixtures, execution, state assertions and result comparison. They are not isolated linking-algorithm benchmarks.

Exact six-run timing records, including warm-ups, are retained in each `timings.csv`.

9. Four append-ready faculty records

Download Information Expert Append-Ready JSON ↗

The JSON contains four records using exactly the eleven required faculty fields:

`pattern_name`, `instance_name`, `language`, `snippet_path`, `source_loc`, `max_function_cc`, `runtime_compiler`, `test_command`, `test_result`, `median_process_ms`, `language_notes`.

All four records use:

- `pattern_name = Information Expert`
- `instance_name = Parsed Email to Trip Record Linking`

They are ready for later aggregation. The final `cross_language.csv` has not been created.

10. Preservation checks and limitations

The checksums of **112 pre-existing Part3 files** were compared against the pre-implementation snapshot. Every one matched, including the frozen Strategy, Observer and Controller material.

The cumulative ZIP passed integrity checks, and generated `.class` files, C++ executables, Python caches and temporary files were excluded.

This example is deliberately in-memory and dependency-free. It does not reproduce Rails, real email parsing, Anthropic, database transactions or Expense creation. The frozen fixture set contains at most one candidate item, and matching is limited to ASCII case-insensitive comparison.

The tests establish behavior for I01–I08, not every possible malformed input or concurrent update.

11. Final Part 3 progress

Pattern	Source ID	Status
Strategy (GoF)	S3-N01	Completed and frozen
Observer (GoF)	P08	Completed and frozen
Controller (GRASP)	P03	Completed and frozen
Information Expert (GRASP)	**P01**	**Completed and verified**

Adapter (GoF)	S2-N01	Not started

16 of the required 20 language implementations are now complete.

P01 — Information Expert is complete for the cross-language coursework stage. DaPlanStan and all previously completed pattern materials remain unchanged. Adapter implementation has not begun.

- **information_expert_append_ready.json**
  Code
- **information_expert_cases.tsv**
  Spreadsheet
- **information_expert_expected.tsv**
  Spreadsheet

Pasted text(20261011-091746).txt
Document

Worked for 6m 8s

Part 3 — Adapter (GoF) Cross-Language Implementation

Completed — 40/40 tests passed

Pattern ID: S2-N01
Pattern: Adapter (GoF)
Confirmed instance: Multiple Booking Provider Normalization

The fifth and final selected pattern has been implemented and tested in Java, Python, JavaScript and C++.

All four implementations use the same frozen A01–A10 fixtures, identical expected outputs and deterministic error codes.

Adapter implementations	Shared Adapter cases passed
4/4	**40/40**
All required languages	Zero failures or errors

1. Download the completed Adapter package

Complete Adapter Cross-Language Package

Cumulative Part3 ZIP · 181 files · Approximately 183 KB

Download Adapter S2-N01 ZIP ↗

Adapter Verification Summary ↗

Four Append-Ready Records ↗

Shared Adapter Cases ↗ · Expected Results ↗

The ZIP passed integrity verification and contains the previous four patterns unchanged.

2. Exact Adapter files created

Plain text

```
Part3/
├── ADAPTER_VERIFICATION_SUMMARY.md
├── common_specs/
│   ├── ADAPTER_README.md
│   ├── adapter_cases.tsv
│   ├── adapter_expected.tsv
│   ├── atlas_raw.tsv
│   ├── beacon_raw.tsv
│   └── adapter_invalid_fixtures/
│       ├── atlas_invalid_category.tsv
│       ├── atlas_invalid_price.tsv
│       ├── atlas_missing_structure.tsv
│       ├── beacon_invalid_type.tsv
│       └── beacon_missing_structure.tsv
├── adapter/
│   ├── adapter_append_ready.json
```

```
│   ├── frozen_before_sha256.json
│   ├── frozen_fixtures_sha256.json
│   ├── java/
│   │   ├── Main.java
│   │   ├── TestMain.java
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── python/
│   │   ├── main.py
│   │   ├── test_main.py
│   │   ├── README.md
│   │   ├── test_result.txt
│   │   └── timings.csv
│   ├── javascript/
│   │   ├── main.js
│   │   ├── test_main.js
```

These are 43 new Adapter-related source, fixture, documentation and evidence files.

3. Frozen A01–A10 fixture validation

The shared inputs and expected results were written and checksummed before implementation.

Case	Scenario	Verified result
A01	Atlas Rome walking sampler	`do` , 2400 cents, EUR
A02	Beacon Rome museum sampler	`do` , 1600 cents, EUR
A03	Atlas Paris market browsing	`shop` , 0 cents
A04	Beacon Paris pastry stop	`eat` , 1250 cents
A05	Unknown provider	`ERROR UNKNOWN_PROVIDER`
A06	Unsupported destination	`ERROR UNKNOWN_DESTINATION`
A07	Atlas selected with Beacon offer ID	`ERROR UNKNOWN_OFFER`

A08	Unsupported Atlas category	`ERROR INVALID_OFFER`
A09	Malformed Atlas compound price	`ERROR INVALID_OFFER`
A10	Missing required Atlas fixture column	`ERROR INVALID_FIXTURE`

All four implementations produced identical outputs matching `adapter_expected.tsv`.

The fixture hashes are stored in `adapter/frozen_fixtures_sha256.json` and `common_specs/ADAPTER_README.md`.

4. Atlas and Beacon normalization

The two provider formats are structurally different.

Property	Atlas Demo	Beacon Demo
Structure	Destination-scoped records	Flat offer records
ID	`id`	`code`
Name	`title`	`label`
Category	`category`	`type`
Location	`location`	`address`
Description	`details`	`summary`
Price	Compound, e.g. `EUR:24.00`	Separate `amount` and `currency`
Destination	First column	Final column

Atlas maps `activity → do`, `shopping → shop` and `dining → eat`.

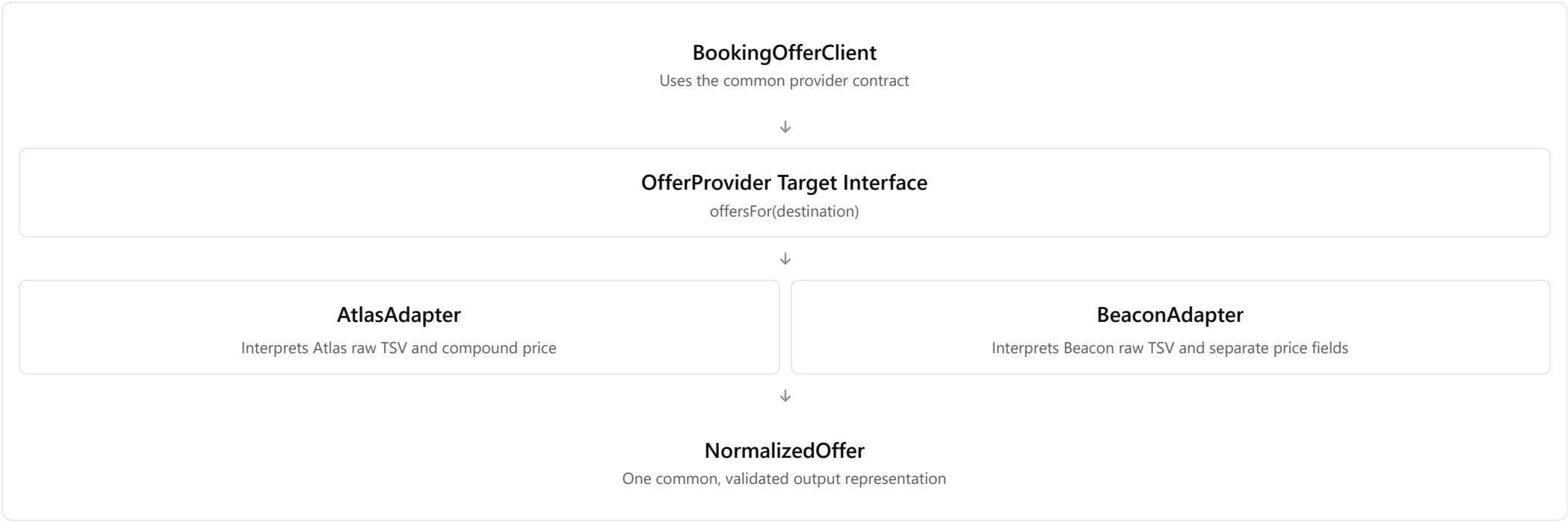
Beacon independently maps `experience → do`, `retail → shop` and `dining → eat`.

Both adapters produce the same normalized fields:

`provider_code`, `provider_name`, `offer_id`, `name`, `kind`, `address`, `description`, `sample_amount_cents`, `currency`.

Amounts are stored as integer cents. Neither adapter creates a booking or confirmation reference.

5. Adapter collaboration and role mapping



Adapter role	Participant in all four implementations
Client	`BookingOfferClient`
Target	`OfferProvider`
ConcreteAdapter 1	`AtlasAdapter`
ConcreteAdapter 2	`BeaconAdapter`
Adaptees	Atlas and Beacon raw TSV formats
Selection	`ProviderRegistry`

Target output	`NormalizedOffer`

Python uses the method name `offers_for`; Java, JavaScript and C++ use `offersFor`.

Why this is Adapter rather than Strategy: The key responsibility is translating incompatible source structures into a common client-facing contract. The two implementations do not represent competing cancellation or booking algorithms.

`ProviderRegistry` is a fixed whitelist and is not counted as a Factory.

6. Actual compilation and test results

Language	Runtime/compiler	Test command	Passed	Failures	Errors	Exit
Java	OpenJDK / javac 21.0.12.1	`java TestMain`	10/10	0	0	0
Python	Python 3.13.5	`python3 test_main.py`	10/10	0	0	0
JavaScript	Node.js v22.16.0	`node test_main.js`	10/10	0	0	0
C++	g++ 14.2.0, C++17	`./adapter_tests`	10/10	0	0	0

Java compilation:

```bash
javac Main.java TestMain.java
java TestMain
```

C++ compilation:

```bash
```

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic main.cpp test_main.cpp -o adapter_tests
./adapter_tests
```

Both compilation commands succeeded. Python and JavaScript required no compilation step.

Total: 40/40 shared cases passed.

7. Provider isolation verification

Additional assertions beyond A01–A10 checked:

- Atlas cannot retrieve Beacon offer IDs.
- Beacon cannot retrieve Atlas offer IDs.
- Rome and Paris filtering works.
- Malformed Atlas data does not prevent valid Beacon lookup.
- Malformed Beacon data does not prevent valid Atlas lookup.
- Invalid Beacon kinds produce `INVALID_OFFER`.
- Missing Beacon schema fields produce `INVALID_FIXTURE`.
- Provider selection does not depend on mutable global state.

These additional isolation checks passed in all four languages. They are not included as additional numbered A01–A10 cases.

8. Production LOC and cyclomatic complexity

Language	Production LOC	Maximum function CC
Java	120	12
Python	93	7
JavaScript	67	8
C++	141	12

LOC excludes tests, blank lines, comments, fixture files and generated artifacts. C++ includes `main.hpp` and `main.cpp`.

Cyclomatic complexity uses the same frozen McCabe-style counting helper used for Strategy, Observer, Controller and Information Expert. Only function discovery was adapted for the new class names and signatures.

These are reproducible source-level approximations, not runtime profiler results.

9. Five measured execution times and medians

Each language completed one warm-up followed by five measured fresh-process runs. Every measured run passed all ten cases and the additional isolation assertions.

All values in milliseconds; medians exclude the warm-up.

Language	Run 1	Run 2	Run 3	Run 4	Run 5	Median
Java	70.547	76.930	66.132	71.668	93.365	**71.668**
Python	865.774	866.613	835.154	852.240	846.508	**852.240**
JavaScript	35.454	38.800	35.246	33.097	34.354	**35.246**
C++	2.522	2.698	2.706	3.606	3.631	**2.706**

Median fresh-process runtime

Complete fixture-based test runner, not isolated adapter performance

These timings include startup, fixture reading, normalization, error handling, isolation assertions and expected-output comparisons. They are not pure Adapter-algorithm benchmarks.

Full-precision values and the warm-up measurement are available in each language's `timings.csv`.

10. Four append-ready faculty records

Download Adapter Append-Ready JSON ↗

The file contains four records using exactly the faculty fields:

`pattern_name`, `instance_name`, `language`, `snippet_path`, `source_loc`, `max_function_cc`, `runtime_compiler`, `test_command`, `test_result`, `median_process_ms`, `language_notes`.

All four use:

- `pattern_name = Adapter`
- `instance_name = Multiple Booking Provider Normalization`

The records contain the actual measured LOC, complexity, compiler/runtime versions, test results and median process times.

The final `cross_language.csv` has deliberately **not** been generated.

11. Preservation verification

Before implementation, the existing Part3 files were checksummed. Afterward, they were checked again.

Preservation check	Result
Existing Part3 files	138/138 unchanged
Previous cumulative ZIP entries	137/137 byte-identical
New Adapter fixture files	9/9 unchanged after freeze

Strategy source and evidence	Preserved
Observer source and evidence	Preserved
Controller source and evidence	Preserved
Information Expert source and evidence	Preserved
DaPlanStan application source	Not modified
Final Adapter ZIP integrity	Passed

The cumulative ZIP includes all five completed patterns, fixtures, append-ready records and verification evidence, excluding compiled binaries and caches.

12. Remaining limitations

This is a dependency-free demonstration of the verified Adapter collaboration. It does not reproduce Rails, Active Record, actual provider APIs or a real booking transaction.

The example is restricted to two fictional providers, two destinations, three normalized kinds and the fixed test inputs. It handles malformed local fixtures deterministically but does not test live network failures, concurrency or production provider changes.

The execution times are machine-specific and heavily influenced by process startup and fixture handling.

13. Final Part 3 implementation progress

Pattern	Source ID	Status
Strategy (GoF)	S3-N01	Completed and frozen
Observer (GoF)	P08	Completed and frozen
Controller (GRASP)	P03	Completed and frozen
Information Expert (GRASP)	P01	Completed and frozen
Adapter (GoF)	**S2-N01**	**Completed and verified**

All 20 required cross-language implementations are now complete.

The remaining stage is **final Part 3 aggregation and cross-language report validation**, using the five frozen append-ready datasets.

No final `cross_language.csv` has been generated, and no other pattern work or DaPlanStan source changes were performed.

- **adapter_append_ready.json** — Code
- **adapter_cases.tsv** — Spreadsheet
- **adapter_expected.tsv** — Spreadsheet

Pasted text(20261011-092718).txt — Document

Worked for 4m 16s

DaPlanStan — Final Cross-Language Aggregation

Aggregation complete and validated

All five confirmed pattern types have been represented in Java, Python, JavaScript and C++. The final faculty dataset, comparative report, validation record and cumulative Part3 package are complete.

5	20	192/192
Pattern types	Implementations	Recorded tests passed

1. Download the final artifacts

Complete Final Part3 Package

186 files · 197 KB · ZIP integrity verified

Download Complete Part3 ZIP ↗

Faculty Cross-Language Dataset

Exactly 20 rows and 11 required columns

Download cross_language.csv ↗

Final Academic Comparison Report

Methodology, five pattern analyses, metric comparisons, verification matrix and limitations

Download CROSS_LANGUAGE_FINAL_REPORT.md ↗

Download CROSS_LANGUAGE_VALIDATION.md ↗

Download Concise Team Report Summary ↗

2. Final CSV validation

I read the actual five frozen append-ready JSON files and compared their records against the shared fixtures, test logs, static metric files and timing evidence.

Validation	Result
Required columns	Exactly 11
Data rows	Exactly 20
Distinct patterns	5
Languages per pattern	4
Duplicate pattern/language records	0
Missing production source paths	0
Missing metric or execution fields	0
Mismatched source instance names	0

Recorded median mismatches	0
Test-output discrepancies	0
Archive integrity	Passed

The CSV preserves the exact recorded values without replacing them with estimates or newly calculated measurements.

Row distribution

Pattern	Java	Python	JavaScript	C++	Total
Strategy	1	1	1	1	4
Observer	1	1	1	1	4
Controller	1	1	1	1	4
Information Expert	1	1	1	1	4
Adapter	1	1	1	1	4
Total	**5**	**5**	**5**	**5**	**20**

3. Consolidated test results

Pattern	Tests per language	Total verified cases
Strategy	15/15	60/60
Observer	7/7	28/28
Controller	8/8	32/32
Information Expert	8/8	32/32
Adapter	10/10	40/40
Total	**48/48**	**192/192**

All 20 recorded execution logs have zero failures, zero errors and exit code 0. Their captured fixture output was checked against the corresponding frozen expected TSV.

The Adapter's extra provider-isolation assertions are recorded separately and have not been used to inflate the 192 counted cases.

These are standalone cross-language tests. The previously blocked DaPlanStan Rails integration tests are a separate matter.

4. Production LOC comparison

Production code only; tests, fixtures, blank lines, comment-only lines and generated artifacts excluded.

Pattern	Java	Python	JavaScript	C++
Strategy	52	54	42	74
Observer	47	32	43	60
Controller	66	59	55	90
Information Expert	63	45	38	83
Adapter	120	93	67	141

C++ counts include both `main.hpp` and `main.cpp`, although the faculty dataset's `snippet_path` convention points to the primary production file, `main.cpp`.

5. Maximum function cyclomatic complexity

Source-level McCabe-style approximations, using the frozen decision-counting rule.

Pattern	Java	Python	JavaScript	C++
Strategy	12	8	8	15
Observer	5	5	5	4
Controller	9	10	9	13
Information Expert	7	7	6	7
Adapter	12	7	8	12

The values are taken from the original per-pattern measurement evidence. They are not formal compiler-generated complexity metrics.

6. Median process runtime comparison

Milliseconds. Each median comes from one warm-up followed by five measured fresh-process runs; compilation excluded.

Pattern	Java	Python	JavaScript	C++
Strategy	91.907	839.469	30.426	2.582
Observer	109.548	1133.245	43.673	2.325
Controller	159.473	818.330	37.862	2.566
Information Expert	75.602	819.622	36.436	4.391
Adapter	71.668	852.240	35.246	2.706

Median process time by language and pattern

Complete test-runner execution, logarithmic differences in magnitude are shown on a linear time axis

The measurements include process startup, fixture parsing, assertions and output comparison. They do **not** measure isolated design-pattern algorithm performance.

7. Pattern-role preservation

Pattern and source ID	Original instance	Preserved cross-language collaboration
Strategy — S3-N01	Cancellation Policy Evaluation	`QuoteEvaluator` selects interchangeable Flexible, Moderate and Strict policies
Observer — P08	Outbound Mail Delivery Logging	`MailDeliveryPublisher` registers/unregisters observers and notifies `MailLogger` through a common callback
Controller — P03	Trip Creation Coordination	`TripCreationController` coordinates Trip, Account, membership, storage and a simulated cover task
Information Expert — P01	Parsed Email to Trip Record Linking	`InboxItem` owns the linking/creation decision using the parsed information it holds
Adapter — S2-N01	Multiple Booking Provider Normalization	Atlas and Beacon adapters translate incompatible inputs through one `OfferProvider` contract into `NormalizedOffer`

All five examples retain the responsibility or collaboration that justified their original pattern classification.

For the GRASP examples in particular, the selected responsibilities remain meaningful outside Rails: the Controller receives and coordinates the system event, while the Information Expert owns behavior based on its information.

8. Cross-language observations

The implementations illustrate different language mechanisms rather than different business requirements.

Java and C++ use explicit interfaces and types extensively. Python employs standard-library abstractions and duck typing where suitable. JavaScript expresses shared contracts through compatible methods and callbacks.

The Adapter and Controller examples generally need more production code because they contain additional participant types, validation and coordination. Observer is comparatively compact across all four languages.

Python and JavaScript often express these examples in fewer production lines than Java and C++, but fewer lines do not automatically imply better maintainability. Similarly, the observed timing differences cannot establish which language is inherently better for a given pattern.

The complete report discusses all five collaborations individually, including C++ observer-pointer lifetime assumptions and the distinction between Adapter's source-format translation and Strategy's interchangeable algorithms.

9. Final verification matrix

The following summarizes the 20 recorded implementations. All have compiler/runtime evidence, measured LOC and CC, one warm-up plus five timing records, and an append-ready record.

Pattern	Language	Fixture tests	Runtime	Metrics	Final status
Strategy	Java	15/15	Verified	Complete	PASS
Strategy	Python	15/15	Verified	Complete	PASS
Strategy	JavaScript	15/15	Verified	Complete	PASS
Strategy	C++	15/15	Verified	Complete	PASS
Observer	Java	7/7	Verified	Complete	PASS
Observer	Python	7/7	Verified	Complete	PASS
Observer	JavaScript	7/7	Verified	Complete	PASS
Observer	C++	7/7	Verified	Complete	PASS
Controller	Java	8/8	Verified	Complete	PASS
Controller	Python	8/8	Verified	Complete	PASS
Controller	JavaScript	8/8	Verified	Complete	PASS
Controller	C++	8/8	Verified	Complete	PASS
Information Expert	Java	8/8	Verified	Complete	PASS
Information Expert	Python	8/8	Verified	Complete	PASS

Information Expert	JavaScript	8/8	Verified	Complete	PASS
Information Expert	C++	8/8	Verified	Complete	PASS
Adapter	Java	10/10	Verified	Complete	PASS
Adapter	Python	10/10	Verified	Complete	PASS
Adapter	JavaScript	10/10	Verified	Complete	PASS
Adapter	C++	10/10	Verified	Complete	PASS

Runtime/compiler versions confirmed from the saved evidence are OpenJDK and javac 21.0.12.1, Python 3.13.5, Node.js v22.16.0, and g++ 14.2.0 using C++17.

Compilation and execution were completed during the preceding pattern stages; this final step independently validated their saved execution evidence rather than rerunning all 20 suites.

10. Preservation and final ZIP validation

The historical cumulative archive chain was checked:

✓ Strategy	34 files
✓ Observer	68 files
✓ Controller	104 files
✓ Information Expert	137 files
✓ Adapter	181 files
✓ Final Part3	186 files

Every file from each historical archive appears byte-identically in the subsequent archive. The 181 Adapter-stage files remain unchanged inside the 186-file final ZIP.

Compiled Java classes, C++ executable binaries, Python caches and temporary runtime files were excluded.

The final ZIP integrity check passed.

11. Remaining limitations

The twenty implementations are deliberately small, fixture-driven demonstrations. They do not reproduce Rails, persistence, Anthropic generation, live geocoding/Turbo, external provider APIs, real mail delivery or booking cancellations.

LOC and complexity are source-based measurements; benchmark medians are specific to the recorded machine and full process workload. Neither should be used alone to rank design quality.

DaPlanStan's full Rails integration/regression testing remains outstanding because the previously available environment used Ruby 3.3.8 instead of the required Ruby 3.4.2 and lacked the locked Rails 8.1.3 dependencies.

12. Remaining manual work before team submission

1. **Faculty-format review:** Confirm the final `cross_language.csv` placement, stable pattern names and repository-relative paths against the faculty instructions.
2. **Independent reproducibility:** Compile and rerun the 20 snippets on the team's submission machine if reproducibility evidence is required; retain actual versions and execution logs.
3. **Rails verification:** Use a compatible Ruby/Rails environment to run the postponed application integration and regression tests. Record actual results without changing the established source-level findings.
4. **Team report assembly:** Integrate the completed evolution/refactoring report with `CROSS_LANGUAGE_FINAL_REPORT.md`, avoiding duplicate or contradictory pattern counts.
5. **Human sign-off:** Verify team contributions, source attribution, historical evidence and the final submission folder contents.

Final completion status

- ✓ **Five pattern types represented across four languages**
- ✓ **20 faculty CSV records validated**
- ✓ **192/192 recorded fixture tests verified**
- ✓ **LOC, cyclomatic complexity and runtime medians preserved**
- ✓ **Final academic report, validation record and ZIP created**
- ✓ **DaPlanStan, previous Part3 implementations and pattern-evolution records unchanged**

Final cross-language aggregation is complete and ready for team-report integration.

CROSS_LANGUAGE_TEAM_SUMMARY.md
Document